

Factorisation matricielle rapide pour la recommandation en ligne

à partir de retours implicites : une implémentation Spark et une étude expérimentale de sa scalabilité

Thomas Sinapi

Anaël Ernadote

M2 IASD - Université Paris-Dauphine PSL
Projet *Machine Learning for Big Data* - 2026

Abstract

He et al. [5] proposent **eALS**, un algorithme de recommandation qui apprend à partir de simples traces d'usage (clics, achats) plutôt que de notes. Sa particularité est de traiter les articles que l'utilisateur n'a pas vus non pas tous de la même façon, mais en fonction de leur popularité. Les auteurs affirment que leur algorithme se parallélise très facilement, sans jamais le vérifier : ils ne fournissent aucune implémentation distribuée.

C'est ce que fait ce rapport. Nous écrivons eALS en PySpark, ainsi qu'une version séquentielle de référence en NumPy, et nous montrons que les deux produisent exactement le même modèle (écart maximal $7 \cdot 10^{-14}$) : la parallélisation ne coûte aucune précision. Nous mesurons ensuite le comportement du code sur un portable à 8 cœurs et quatre jeux de données de 10^5 à 10^7 interactions, dont Amazon Movies, l'un des deux jeux utilisés par l'article.

Deux résultats principaux. Côté précision, eALS obtient +35 % de HR@100 par rapport à l'ALS de Spark MLlib, et son coût croît beaucoup plus lentement avec le nombre de facteurs K ($2.2\times$ contre $10.3\times$ entre $K = 32$ et $K = 128$). Côté parallélisme en revanche, l'accélération plafonne à $2.07\times$ sur 8 cœurs : la limite n'est pas l'algorithme, mais la machine.

Contents

1	Introduction et énoncé du problème	2
2	La solution retenue	3
2.1	Architecture générale	3
2.2	RDD ou DataFrame ?	3
2.3	Choix du jeu de données	4
2.4	Matériel et logiciels	4
3	L'algorithme	5
3.1	Modèle et objectif	5
3.2	Dériver la mise à jour élément par élément	5
3.3	Complexité	7
3.4	Distribuer une itération	7
4	Commentaire sur les principaux fragments de code	8
4.1	Le noyau de calcul	8
4.2	Construire les blocs une seule fois	9
4.3	Diffusion du modèle et localisation de l'état	10
4.4	Croissance du plan d'exécution dans une boucle Spark	10
4.5	Environnement d'exécution des processus distants	11
4.6	Évaluer sans construire la matrice des scores	11

5	Analyse expérimentale	12
5.1	Exactitude de la parallélisation	12
5.2	Scalabilité	13
5.2.1	Scalabilité forte	13
5.2.2	Scalabilité en nombre de facteurs latents	14
5.3	Convergence et précision	16
6	Discussion : forces et faiblesses	18
6.1	Points forts	18
6.2	Points faibles	19
6.3	Limites de validité des mesures	20
7	Conclusion et travaux futurs	21
	Déclaration sur l'usage d'outils d'assistance	22
A	Code source complet	23
A.1	src/spark_utils.py - session Spark	23
A.2	src/data.py - chargement, filtrage 10-core, découpage	23
A.3	src/eals_local.py - référence NumPy et noyaux partagés	25
A.4	src/eals_rdd.py - implémentation Spark RDD	27
A.5	src/evaluate.py - HR@k, NDCG@k, classement sur catalogue entier	30
A.6	src/baselines.py - ALS de MLlib et MostPopular	31
A.7	src/check_correctness.py - le contrôle d'exactitude	31
A.8	src/experiments.py - la campagne de mesures	33
A.9	src/plots.py - figures	36
A.10	report/tables.py - chaque chiffre de ce rapport	38
A.11	scripts/env.sh, run_all.sh, build_report.sh	42
B	Le notebook de démonstration	43

1 Introduction et énoncé du problème

Le problème. Un système de recommandation apprend rarement à partir de notes. Il apprend à partir de traces : ce que l'utilisateur a cliqué, lu, acheté. Ces traces sont abondantes, mais elles n'ont qu'un seul côté. Elles disent ce qu'un utilisateur a fait, jamais ce qu'il a rejeté. Un article non consulté peut aussi bien signifier « ça ne m'intéresse pas » que « je ne l'ai jamais vu passer ».

C'est une vraie difficulté, pas un détail. Si l'on entraîne un modèle uniquement sur les interactions observées, il apprend à répondre « oui » partout, et ce modèle absurde obtient un score parfait : il n'a jamais vu un seul exemple négatif. La solution classique, due à Hu et al. [6], consiste à considérer que toute paire (utilisateur, article) non observée est un négatif, mais un négatif *faible*, auquel on accorde un poids w_0 petit et identique partout.

Ce que change eALS. L'article [5] conteste ce poids unique. Ne pas avoir vu un film à succès est un indice assez fort de désintérêt ; ne pas avoir vu un film confidentiel n'indique presque rien. Les auteurs remplacent donc le poids unique w_0 par un poids c_i propre à chaque article, croissant avec sa popularité.

Le problème est que ce changement, apparemment anodin, casse l'algorithme existant. L'ALS classique calcule un vecteur entier de préférences à la fois, ce qui l'oblige à inverser une matrice pour chaque utilisateur et chaque article. Cette opération n'est abordable que grâce à un précalcul partagé, et ce précalcul n'est possible que si le poids est identique partout. Autrement dit : dès que l'on veut des poids différenciés, la recette ne fonctionne plus.

La réponse de He et al. est de changer de granularité. Plutôt que de résoudre un vecteur entier, ils mettent à jour *une seule coordonnée à la fois*. Chaque mise à jour devient alors une simple division, sans aucune inversion de matrice. C’est ce que signifie le « e » d’eALS : *element-wise*, élément par élément. Le gain est d’un facteur K (le nombre de dimensions du modèle) sur le coût d’une itération, et les poids peuvent enfin varier d’un article à l’autre.

Le point de départ de ce projet. Ces mises à jour ne lisent que des données partagées en lecture seule. Les auteurs en concluent qu’eALS est « trivialement parallélisable » : on peut confier des groupes d’utilisateurs différents à des machines différentes, et le résultat sera *exactement* celui du calcul séquentiel, pas une approximation.

L’article s’arrête là. Aucune implémentation distribuée n’est fournie, et cette affirmation n’est jamais vérifiée. C’est précisément ce que demande le sujet du projet, et ce que fait ce rapport : écrire l’implémentation Spark manquante, vérifier que le parallélisme est bien exact, et mesurer ce qu’il rapporte réellement.

Plan. La Section 2 décrit la solution retenue et justifie les choix de conception. La Section 3 explique l’algorithme et comment nous le distribuons. La Section 4 commente les fragments de code qui comptent. La Section 5 présente les mesures. La Section 6 discute forces et faiblesses. Les annexes contiennent le code complet et un notebook de démonstration.

2 La solution retenue

2.1 Architecture générale

Le système est un petit paquet Python plat (Table 1). Il n’y a délibérément ni framework, ni hiérarchie de classes, ni couche de configuration : ce qui est intéressant, c’est le noyau numérique et la façon dont Spark l’ordonne, et les deux doivent tenir sur un écran.

Table 1: Modules. Tout est en Python ; NumPy fait l’algèbre locale, Spark fait la distribution.

Module	Rôle	Lignes
<code>spark_utils.py</code>	l’unique endroit où la <code>SparkSession</code> est configurée	42
<code>data.py</code>	chargement, filtrage 10-core, ré-indexation dense, découpage LOO	133
<code>eals_local.py</code>	référence NumPy <i>et</i> les deux noyaux de mise à jour partagés	147
<code>eals_rdd.py</code>	implémentation Spark RDD (fondée sur les broadcasts)	173
<code>baselines.py</code>	ALS implicite de MLlib [9], MostPopular	37
<code>evaluate.py</code>	classement sur le catalogue entier, $HR@k$, $NDCG@k$	69
<code>check_correctness.py</code>	les deux implémentations doivent concorder	99
<code>experiments.py</code>	chaque mesure → <code>results/*.csv</code>	217
<code>plots.py</code>	chaque figure → <code>results/figures</code>	158

Une propriété que nous jugeons importante : **le noyau numérique n’existe qu’une seule fois**. `eals_local.update_users` et `update_items` sont importées telles quelles par la version RDD. Seul l’*ordonnancement* diffère entre les deux. Tout écart entre elles localise donc un bug de plomberie, jamais un bug de formule - et la Section 5.1 montre qu’il n’y en a aucun.

2.2 RDD ou DataFrame ?

L’énoncé autorise l’un ou l’autre. Nous avons retenu les **RDD [11] avec broadcast**, et le choix se justifie par un argument de volume de communication plutôt que par une préférence d’API. Les deux parallélisations envisageables ne sont pas deux écritures d’un même programme :

- **RDD + broadcast** (retenu). Les matrices de facteurs vivent sur le driver ; à chaque étape, celle qui est en lecture seule est diffusée (broadcast), et chaque tâche met à jour les lignes

qu'elle possède. Par étape, le réseau transporte $O(NK)$ (le broadcast) plus $O(MK)$ (le résultat collecté).

- **DataFrame + jointure/cogroup.** Les facteurs restent distribués sous forme de DataFrames ; une tâche reçoit les facteurs dont elle a besoin via une jointure avec shuffle. Rien n'est jamais assemblé sur le driver, donc le modèle peut dépasser sa mémoire - mais le shuffle transporte $O(|\mathcal{R}|K)$ par étape au lieu de $O((M+N)K)$. Sur ml-1m ce rapport vaut $|\mathcal{R}|/N \approx 300$.

Sur la machine visée - un seul portable, un modèle de quelques dizaines de Mo - la contrainte de mémoire du driver qui justifierait la seconde option ne se présente jamais, tandis que le facteur ≈ 300 sur le volume échangé, lui, se paie à chaque itération. Nous implémentons donc uniquement la version RDD. La limite de cette décision est discutée en Section 6 : c'est elle qui borne la taille de modèle que ce code peut traiter.

2.3 Choix du jeu de données

Nous utilisons deux familles de jeux de données, pour deux usages distincts.

Amazon Movies est l'un des deux jeux de l'article, et c'est là que nous mesurons la précision. Il est très creux (99,87 % de cases vides), ce qui correspond au régime d'un vrai catalogue. Nous partons du fichier de notes de la catégorie Films & TV du graphe produits Amazon [8], et non du dump brut cité par l'article, qui n'est plus commode à obtenir ; c'est la même source et le même domaine, sur un instantané un peu différent. Nos chiffres de précision ne sont donc pas directement comparables à ceux de l'article, et nous ne le prétendons pas.

MovieLens 100k / 1M / 10M [4] sert aux mesures de temps. Une étude de scalabilité a besoin du même jeu de données à plusieurs tailles, ce qu'Amazon ne fournit pas : MovieLens offre une échelle propre en $\times 100$ avec une sémantique identique à chaque palier.

Les quatre jeux subissent le même prétraitement : les notes sont ignorées (une interaction vaut 1), on ne garde que les utilisateurs et les articles ayant au moins 10 interactions (filtrage *10-core*), les identifiants sont renumérotés de façon contiguë, et l'on met de côté la dernière interaction de chaque utilisateur pour le test (protocole *leave-one-out*).

Table 2: Jeux de données après filtrage 10-core et ré-indexation dense.

Jeu de données	Utilisateurs M	Items N	$ \mathcal{R} $	Sparsité	Train	Test LOO
ml-100k	943	1 152	97 953	90.98%	97 010	943
ml-1m	6 040	3 260	998 539	94.93%	992 499	6 040
ml-10m	69 878	9 708	9 995 471	98.53%	9 925 593	69 878
amazon-movies	33 326	21 901	958 986	99.87%	925 660	33 326

2.4 Matériel et logiciels

Tout tourne sur un unique **ordinateur portable Apple série M, 8 cœurs physiques, 16 Go**, avec **Spark 4.1.0** en mode `local [p]`, **Python 3.11**, **NumPy 2.3** et **OpenJDK 17**. Il n'y a pas de cluster : p , le nombre de cœurs, est le réglage que font varier les expériences de parallélisme.

Un point de configuration mérite d'être signalé car il conditionne la validité des mesures. Chaque tâche Spark appelle NumPy, et NumPy lance de lui-même ses propres threads de calcul. Sur 8 cœurs, cela donne 8 tâches $\times$ 8 threads = 64 threads en concurrence, et la mesure à « 1 cœur » se retrouve silencieusement parallélisée : toute accélération calculée ensuite est dénuée de sens. Nous bridons donc NumPy à un seul thread (`scripts/env.sh`), de sorte que Spark soit la seule source de parallélisme. Signalons aussi que Spark 4 exige Java 17 ou 21 ; sur un JDK plus récent, aucun job ne démarre.

Tous les temps rapportés sont des médianes. Chaque configuration est répétée 3 fois, chaque exécution effectue 4 itérations dont la première est écartée (échauffement), soit 9 mesures par point ; les barres d'erreur des figures donnent le minimum et le maximum. La graine aléatoire est fixée partout, si bien que deux exécutions identiques produisent le même modèle au bit près.

3 L'algorithme

3.1 Modèle et objectif

L'idée du modèle. On dispose d'un tableau géant à M lignes (les utilisateurs) et N colonnes (les articles), rempli de 1 là où une interaction a eu lieu et de 0 partout ailleurs. Ce tableau est immense et presque vide : sur Amazon Movies, plus de 99,8 % des cases sont à zéro.

La factorisation matricielle consiste à décrire chaque utilisateur et chaque article par une courte liste de K nombres, appelés *facteurs latents* - typiquement $K = 32$. On peut se les représenter comme des goûts non nommés : « films d'action », « cinéma d'auteur », etc., découverts automatiquement plutôt que définis à la main. La note prédite pour un couple (utilisateur, article) est alors simplement le produit scalaire de leurs deux listes : si les goûts coïncident, le score est élevé.

Apprendre le modèle revient donc à chercher les listes de nombres qui reproduisent au mieux le tableau observé. Formellement, avec P la matrice des facteurs utilisateurs, Q celle des articles et $\hat{r}_{ui} = \mathbf{p}_u^\top \mathbf{q}_i$ la prédiction, on minimise l'écart quadratique sur *toutes* les cases, chacune pondérée par un poids w_{ui} (Éq. 2 de l'article) :

$$J = \sum_{u=1}^M \sum_{i=1}^N w_{ui} (r_{ui} - \hat{r}_{ui})^2 + \lambda \left(\sum_u \|\mathbf{p}_u\|^2 + \sum_i \|\mathbf{q}_i\|^2 \right), \quad (1)$$

avec $r_{ui} = 1$, $w_{ui} = 1$ sur les paires observées et $r_{ui} = 0$, $w_{ui} = c_i$ ailleurs. Le second terme, piloté par λ , est une régularisation : il pénalise les facteurs de grande amplitude et empêche le modèle de surapprendre. En séparant les cases observées des cases vides, on obtient la forme réellement optimisée (Éq. 7) :

$$J = \sum_{(u,i) \in \mathcal{R}} w_{ui} (r_{ui} - \hat{r}_{ui})^2 + \sum_{u=1}^M \sum_{i \notin \mathcal{R}_u} c_i \hat{r}_{ui}^2 + \lambda \left(\sum_u \|\mathbf{p}_u\|^2 + \sum_i \|\mathbf{q}_i\|^2 \right). \quad (2)$$

Le poids des articles non vus (Éq. 8). C'est ici que se situe l'apport de l'article. Le poids accordé à une case vide de la colonne i vaut

$$c_i = c_0 \frac{f_i^\alpha}{\sum_{j=1}^N f_j^\alpha}, \quad f_i = \frac{|\mathcal{R}_i|}{\sum_j |\mathcal{R}_j|}, \quad (3)$$

où f_i est la fréquence de l'article i . La formule a une propriété commode : quelle que soit la valeur de α , la somme des c_i vaut toujours c_0 . Les deux paramètres ont donc des rôles bien séparés. c_0 fixe l'importance *totale* accordée aux cases vides, et α décide seulement de la façon de la répartir entre les articles.

Deux cas limites éclairent le mécanisme. Avec $\alpha = 0$, tous les articles reçoivent le même poids et l'on retrouve exactement la méthode de Hu et al. Avec $\alpha > 0$, les articles populaires en reçoivent davantage : ne pas avoir vu un film à succès pèse plus lourd dans l'apprentissage que ne pas avoir vu un film obscur. Sur Amazon Movies, le rapport entre le plus fort et le plus faible poids passe de 1 à $\alpha = 0$ à environ 16 à $\alpha = 0,4$, la valeur retenue par l'article. Cette redistribution fait tout l'intérêt de la méthode, et aussi, comme nous le verrons en Section 6, tout son risque.

3.2 Dériver la mise à jour élément par élément

Cette sous-section refait le calcul qui mène à la règle de mise à jour. Elle mérite d'être suivie, car c'est elle qui fait apparaître l'astuce sur laquelle repose toute l'efficacité de la méthode - et, plus loin, toute la parallélisation.

Le principe est simple : on fixe tous les paramètres sauf un, et on cherche la valeur optimale de celui-là. Comme la fonction à minimiser est un polynôme du second degré en ce paramètre, il suffit d'annuler la dérivée pour obtenir directement l'optimum. Pas de pas d'apprentissage, pas de tâtonnement.

Notons $\hat{r}_{ui}^f = \hat{r}_{ui} - p_{uf}q_{if}$ la prédiction *privée de la contribution du facteur f*. En dérivant (1) par rapport à la seule coordonnée p_{uf} :

$$\frac{\partial J}{\partial p_{uf}} = -2 \sum_{i=1}^N (r_{ui} - \hat{r}_{ui}^f) w_{ui} q_{if} + 2 p_{uf} \sum_{i=1}^N w_{ui} q_{if}^2 + 2 \lambda p_{uf}. \quad (4)$$

En l'annulant on obtient la solution élément par élément générique (Éq. 5)

$$p_{uf} = \frac{\sum_{i=1}^N (r_{ui} - \hat{r}_{ui}^f) w_{ui} q_{if}}{\sum_{i=1}^N w_{ui} q_{if}^2 + \lambda}, \quad (5)$$

Cette formule est exacte, mais inutilisable telle quelle : les sommes portent sur les N articles du catalogue, pour *chaque* utilisateur et *chaque* facteur. On parcourrait donc tout le tableau vide à chaque itération, soit $O(MNK)$ opérations. Sur Amazon Movies, cela représente 23 milliards de termes par itération, alors que seules 10^6 cases contiennent une information.

Toute l'astuce consiste à ne jamais parcourir ces cases vides. Séparons la somme en deux parties, les cases observées et les autres, en utilisant $r_{ui} = 0$ et $w_{ui} = c_i$ sur les secondes :

$$p_{uf} = \frac{\sum_{i \in \mathcal{R}_u} (r_{ui} - \hat{r}_{ui}^f) w_{ui} q_{if} - \sum_{i \notin \mathcal{R}_u} \hat{r}_{ui}^f c_i q_{if}}{\sum_{i \in \mathcal{R}_u} w_{ui} q_{if}^2 + \sum_{i \notin \mathcal{R}_u} c_i q_{if}^2 + \lambda}. \quad (6)$$

Les deux sommes sur les cases vides ($i \notin \mathcal{R}_u$) restent le goulot d'étranglement. L'idée décisive est de les réécrire par complément : *tous les articles, moins ceux que l'utilisateur a vus*. Le premier terme ne dépend alors plus de l'utilisateur, et le second ne porte que sur les quelques articles réellement observés. Pour le numérateur, en développant $\hat{r}_{ui}^f = \sum_{k \neq f} p_{uk} q_{ik}$:

$$\begin{aligned} \sum_{i \notin \mathcal{R}_u} \hat{r}_{ui}^f c_i q_{if} &= \sum_{i=1}^N c_i q_{if} \sum_{k \neq f} p_{uk} q_{ik} - \sum_{i \in \mathcal{R}_u} \hat{r}_{ui}^f c_i q_{if} \\ &= \sum_{k \neq f} p_{uk} \underbrace{\sum_{i=1}^N c_i q_{if} q_{ik}}_{s_{kf}^q} - \sum_{i \in \mathcal{R}_u} \hat{r}_{ui}^f c_i q_{if}, \end{aligned} \quad (7)$$

et pour le dénominateur, $\sum_{i \notin \mathcal{R}_u} c_i q_{if}^2 = s_{ff}^q - \sum_{i \in \mathcal{R}_u} c_i q_{if}^2$. L'observation cruciale est que la matrice $K \times K$

$$S^q = \sum_{i=1}^N c_i \mathbf{q}_i \mathbf{q}_i^\top \quad (8)$$

ne dépend pas de l'utilisateur u . C'est le point clé de toute la méthode : cette petite matrice $K \times K$ (soit 32×32 en pratique) résume à elle seule la contribution des millions de cases vides. On la calcule *une seule fois* par itération, puis on la réutilise pour les M utilisateurs. Le coût des données manquantes passe ainsi de « proportionnel au catalogue » à « constant ».

En reportant (7) dans (6), on obtient la règle effectivement implémentée :

$$p_{uf} \leftarrow \frac{\sum_{i \in \mathcal{R}_u} [w_{ui} r_{ui} - (w_{ui} - c_i) \hat{r}_{ui}^f] q_{if} - \sum_{k \neq f} p_{uk} s_{kf}^q}{\sum_{i \in \mathcal{R}_u} (w_{ui} - c_i) q_{if}^2 + s_{ff}^q + \lambda} \quad (9)$$

Aucune somme ne porte plus sur les N articles : elles portent sur les interactions réelles de l'utilisateur, ou sur les K facteurs. C'est ce qui rend la méthode utilisable.

Côté article. Le calcul est symétrique, avec une différence utile : le poids c_i ne dépend que de l'article, il peut donc sortir de la somme. Avec $S^p = P^\top P$ le second cache (non pondéré, cette fois) :

$$q_{if} \leftarrow \frac{\sum_{u \in \mathcal{R}_i} [w_{ui} r_{ui} - (w_{ui} - c_i) \hat{r}_{ui}^f] p_{uf} - c_i \sum_{k \neq f} q_{ik} s_{kf}^p}{\sum_{u \in \mathcal{R}_i} (w_{ui} - c_i) p_{uf}^2 + c_i s_{ff}^p + \lambda} \quad (10)$$

C'est l'Éq. (13) de l'article.

Cette réécriture est une optimisation, et une optimisation peut être fausse. Avant d'écrire la moindre ligne de Spark, nous avons donc comparé les deux règles encadrées à la formule naïve (5), évaluée littéralement sur *toutes* les cases d'un problème jouet de taille 20×15 - assez petit pour que le calcul brut soit possible. L'écart maximal est de l'ordre de 10^{-16} , c'est-à-dire la précision d'un nombre flottant. La mise en cache est donc une réécriture exacte, pas une approximation.

Suivre la convergence sans tout recalculer. Le même problème se pose pour vérifier que le modèle progresse : évaluer (2) directement coûterait à nouveau $O(MNK)$. La même astuce s'applique (Éq. 14 de l'article) :

$$\sum_u \sum_{i \notin \mathcal{R}_u} c_i \hat{r}_{ui}^2 = \sum_{u=1}^M \mathbf{p}_u^\top S^q \mathbf{p}_u - \sum_{(u,i) \in \mathcal{R}} c_i \hat{r}_{ui}^2, \quad (11)$$

ce qui ramène le coût à $O(|\mathcal{R}| + MK^2)$, donc au même ordre qu'une itération. C'est ce que calcule `eals_local.objective`, et c'est ainsi que nous vérifions la convergence.

3.3 Complexité

Table 3: Coût d'une itération. $|\mathcal{R}|$ est le nombre d'interactions observées, M le nombre d'utilisateurs, N celui d'articles et K celui de facteurs.

Méthode	Temps par itération	Poids des cases vides
ALS, Hu et al. [6]	$O((M + N)K^3 + \mathcal{R} K^2)$	uniforme seulement
Formule naïve (5)	$O(MNK)$	quelconque
eALS [5]	$O((M + N)K^2 + \mathcal{R} K)$	un poids c_i par article

Le décompte est direct. Pour un utilisateur donné et un facteur donné, le terme faisant intervenir le cache coûte $O(K)$ et les sommes sur les interactions observées coûtent $O(|\mathcal{R}_u|)$. En sommant sur les K facteurs et les M utilisateurs, une étape revient à $O(MK^2 + |\mathcal{R}|K)$, et le calcul des caches ajoute $O((M + N)K^2)$.

La conclusion pratique tient en une phrase : eALS est un facteur K moins cher qu'ALS, et son coût doit croître comme le *carré* du nombre de facteurs plutôt que comme son *cube*. C'est une prédiction vérifiable, et la Section 5.2.2 la met à l'épreuve.

3.4 Distribuer une itération

Trois choix de conception méritent d'être détaillés.

La parallélisation ne perd aucune précision. C'est l'affirmation centrale de l'article, et elle se justifie simplement. Pendant l'étape utilisateur, Q et le cache S^q ne sont que *lus* ; les seules valeurs *écrites* par un bloc sont les facteurs de ses propres utilisateurs, auxquels aucun autre bloc ne touche. Il n'y a donc jamais deux machines qui modifient la même case.

Le résultat distribué est par conséquent identique au résultat séquentiel, à l'ordre d'addition des nombres flottants près. C'est une propriété rare : la descente de gradient stochastique distribuée, par exemple, ne l'a pas, car des mises à jour concurrentes s'y écrasent mutuellement. La Section 5.1 vérifie l'affirmation numériquement.

```

entrée : P, Q sur le driver ; c diffuse une fois pour toutes
        U_1..U_B partitionnent les utilisateurs, I_1..I_B
        partitionnent les items

0  avant la boucle :  S^q <- somme_i c_i q_i q_i^T ;  bcP <-
    broadcast(P)
    (les seules fois ou ces deux objets sont calcules hors des taches)

1  bcQ <- broadcast(Q)
2  pour chaque bloc b, EN PARALLELE :                                # etape
    utilisateur
3      P_b <- lignes U_b de bcP
4      pour f = 1..K :  mettre a jour p_uf pour TOUS les u de U_b  --
        Eq.(12), vectorise sur u
5          emettre (U_b, P_b, P_b^T P_b)
6  collecter ; disperser les P_b dans P ;  S^p <- somme_b P_b^T P_b
    # cache, gratuit
7  bcP <- broadcast(P)
8  pour chaque bloc b, EN PARALLELE :                                # etape
    item
9      Q_b <- lignes I_b de bcQ
10     pour f = 1..K :  mettre a jour q_if pour TOUS les i de I_b  --
        Eq.(13), vectorise sur i
11         emettre (I_b, Q_b, somme_{i in I_b} c_i q_i q_i^T)
12 collecter ; disperser les Q_b dans Q ;  S^q <- somme_b (.)  #
    cache pour l'iteration suivante

```

Figure 1: Une itération eALS distribuée (eals_rdd.fit). Les lignes 5 et 11 sont ce qui fait que les deux caches S de l’Algorithme 1 de l’article ne coûtent rien : chaque tâche de mise à jour renvoie la matrice de Gram des lignes qu’elle vient d’écrire.

Les caches sont obtenus gratuitement. Une implémentation naïve recalculerait S^q et S^p en parcourant à nouveau toutes les lignes de facteurs, soit deux passes supplémentaires par itération. Nous procédons autrement : chaque tâche renvoie, en même temps que ses facteurs mis à jour, la contribution partielle des lignes qu’elle vient d’écrire (lignes 5 et 11 du pseudo-code). Le driver n’a plus qu’à additionner ces morceaux. Le cache est donc un sous-produit du calcul, et le surcoût réseau est négligeable : quelques matrices $K \times K$, soit environ 131 ko par étape.

Le parallélisme est entre utilisateurs, pas entre facteurs. La boucle sur les K facteurs ne peut pas être parallélisée : le facteur $f + 1$ doit voir la valeur mise à jour du facteur f , sinon on ne résout plus le bon problème. En revanche, rien n’empêche de traiter le facteur f de *tous* les utilisateurs d’un bloc en même temps, puisqu’ils sont indépendants. C’est exactement ce que fait notre noyau, et cela change l’échelle du problème : la boucle Python s’exécute K fois par étape au lieu de $M \times K$ fois. La Section 4 montre le code correspondant.

4 Commentaire sur les principaux fragments de code

Cette section commente les quelques fragments de code où se joue quelque chose de non trivial. Le code complet figure en Annexe A.

4.1 Le noyau de calcul

C’est le cœur du projet : une vingtaine de lignes qui appliquent la règle (9) non pas à un utilisateur, mais à un bloc entier d’utilisateurs d’un seul coup.

```

1 def update_users(PT, QT, idx, rowid, cvec, Sq, lam, w=1.0, rhat=None):

```



```

2 K, m = PT.shape
3 c_e = cvec[idx]
4 wmc = w - c_e # (w_ui - c_i)
5 wr = w * 1.0 # w_ui * r_ui, r_ui = 1
6 if rhat is None:
7     rhat = _predict(PT, QT, rowid, idx)
8 for f in range(K):
9     qf = QT[f][idx]
10    rhat -= PT[f][rowid] * qf # r_hat^f = r_hat - p_u f q_i f
11    num = np.bincount(rowid, (wr - wmc * rhat) * qf, minlength=m)
12    den = np.bincount(rowid, wmc * qf * qf, minlength=m)
13    num -= Sq[:, f] @ PT - PT[f] * Sq[f, f] # - sum_{k!=f} p_u k s^q_k f
14    den += Sq[f, f] + lam
15    PT[f] = num / den
16    rhat += PT[f][rowid] * qf
17 return rhat

```

Listing 1: `eals_local.update_users` - Éq. (9), vectorisée sur les utilisateurs d'un bloc.

- **Le code suit la formule terme à terme.** `num` est le numérateur de (9) et `den` son dénominateur. La ligne `Sq[:,f] @ PT - PT[f]*Sq[f,f]` calcule la somme sur les facteurs autres que f : on somme sur tous les facteurs, puis on retranche le terme en trop, ce qui est plus rapide que de construire une matrice trouée.
- **`np.bincount` fait tout le travail.** C'est la fonction qui permet de calculer, en une seule passe, une somme séparée pour chaque utilisateur du bloc. C'est elle qui rend possible le traitement simultané de tout un bloc, au lieu d'une boucle Python par utilisateur.
- **Les facteurs sont stockés transposés.** La boucle interne accède à un facteur entier à la fois. En rangeant la matrice dans l'autre sens, chaque accès serait dispersé en mémoire au lieu d'être contigu, ce qui coûte cher sur des tableaux de cette taille.
- **Les prédictions sont mises à jour, jamais recalculées.** Le tableau `rhat` contient les prédictions courantes. À chaque facteur, on retire sa contribution (`rhat -= ...`), on l'utilise, puis on rajoute la nouvelle (`rhat += ...`). Sans cette mise à jour incrémentale, il faudrait recalculer chaque prédiction depuis zéro, ce qui multiplierait le coût par K .
- **Aucun grand tableau intermédiaire n'est créé.** Rassembler d'un coup les facteurs de toutes les interactions demanderait 2,5 Go sur ml-10m. Nous récupérons une colonne à la fois, ce qui ramène le besoin mémoire au nombre d'interactions, sans changer le volume total de calcul.

4.2 Construire les blocs une seule fois

```

1 def build_blocks(df, key, val, n_parts):
2     def to_arrays(it):
3         ks, vs = [], []
4         for k, v in it:
5             ks.append(k); vs.append(v)
6         ks = np.asarray(ks, dtype=np.int64); vs = np.asarray(vs, dtype=np.int64)
7         ids, rowid = np.unique(ks, return_inverse=True)
8         o = np.argsort(rowid, kind="stable")
9         return iter([(ids, rowid[o].astype(np.int64), vs[o])])
10
11    rdd = df.select(key, val).rdd.map(lambda r: (r[0], r[1]))
12    return rdd.partitionBy(n_parts).mapPartitions(to_arrays, preservesPartitioning=True)

```

Listing 2: `eals_rdd.build_blocks` - un élément RDD par partition, contenant le bloc sous forme de tableaux NumPy.

- **Les données ne sont réparties qu'une seule fois.** On regroupe les interactions par utilisateur une fois pour toutes, et on garde le résultat en mémoire. Refaire ce regroupement à chaque itération obligerait Spark à redéplacer toutes les interactions sur le réseau à chaque fois. Sur ml-10m, c'est la différence entre une itération de 4 secondes et une de 40.

- **Un bloc entier tient dans un seul objet.** Chaque partition est réduite à trois tableaux NumPy, plutôt qu'à un objet Python par utilisateur. Sur ml-10m, la seconde option ferait payer le coût des objets Python 70 000 fois par étape, contre une poignée de fois ici.
- **Les interactions sont triées à la construction.** Le tri rend les accès mémoire séquentiels dans la boucle de calcul, au lieu de sauter au hasard dans le tableau des facteurs.
- **Les utilisateurs les plus actifs sont répartis entre les blocs.** Les identifiants sont attribués par fréquence décroissante, de sorte que la répartition par défaut disperse les gros utilisateurs au lieu de les entasser dans le même bloc. L'enjeu est réel : sur ml-10m, l'utilisateur le plus actif a 7 063 interactions contre 68 pour l'utilisateur médian. Un bloc qui en concentrerait plusieurs ferait attendre tous les autres.

4.3 Diffusion du modèle et localisation de l'état

```

1 bcQ = sc.broadcast(QT)
2 res = ublocks.map(lambda b: _user_task(b, bcP, bcQ, bcC, lam, w, Squ)).collect()
3 Sp = np.zeros((K, K))
4 for ids, blk, sp in res:
5     PT[:, ids] = blk          # disperser le bloc dans la matrice de facteurs complète
6     Sp += sp                 # ... et accumuler le cache S^p gratuitement
7 bcP.unpersist(); bcP = sc.broadcast(PT)
8 res = iblocks.map(lambda b: _item_task(b, bcP, bcQ, bcC, lam, w, Sp)).collect()

```

Listing 3: eals_rdd.fit - le corps d'une itération.

- **Deux diffusions par itération, pas quatre.** La matrice Q est envoyée une fois en début d'itération et sert aux *deux* étapes ; P est envoyée après l'étape utilisateur et resservira à l'itération suivante. Le détail compte, car chaque machine doit décompresser sa copie : une diffusion inutile se paie autant de fois qu'il y a de processus.
- **Libérer explicitement les copies diffusées.** Spark ne récupère pas automatiquement la mémoire d'une variable diffusée lorsqu'on cesse de s'en servir. Sans l'appel explicite à `unpersist`, le driver accumule une copie de P par itération et finit par saturer.
- **Le modèle ne vit pas dans Spark.** Les facteurs sont un simple tableau NumPy côté driver ; Spark ne sert qu'à distribuer le calcul, pas à stocker l'état. C'est un choix délibéré, et il évite le piège décrit au fragment suivant.
- **C'est aussi la limite de la conception.** Le modèle entier doit tenir sur le driver, et chaque processus de calcul doit pouvoir en garder une copie. À 10^8 utilisateurs et $K = 128$, cela ferait 100 Go et l'approche s'effondrerait. La réponse à cette échelle serait le schéma par jointures évoqué en Section 2.

4.4 Croissance du plan d'exécution dans une boucle Spark

Spark ne calcule rien tant qu'on ne lui demande pas de résultat : il se contente d'enregistrer la recette des opérations. C'est efficace, sauf dans une boucle, où la recette s'allonge à chaque tour. La boucle d'entraînement y échappe, puisque le modèle n'y est pas stocké. La préparation des données, elle, n'y échappe pas : le filtrage 10-core est une boucle de jointures, et c'est là que le problème s'est manifesté.

```

1 nxt = df.join(ok_u, "user").join(ok_i, "item").select("user", "item", "ts")
2 # localCheckpoint, pas cache : cache() garde le plan logique vivant, donc apres ~8
3 # tours le plan est assez profond pour que Spark meure en *affichant* le plan
4 # (TreeNode.generateTreeString). Le checkpoint tronque la lignee en un simple scan
5 # des lignes materialisees.
6 nxt = nxt.localCheckpoint(eager=True)
7 df.unpersist()
8 df = nxt

```

Listing 4: data.kcore - la ligne qui permet à une boucle de jointures Spark de survivre.

- **Mettre en cache ne suffit pas.** On pourrait croire qu'un `cache()` règle le problème. Il matérialise bien les *données*, mais Spark conserve la recette complète, au cas où il devrait tout recalculer. Elle continue donc de s'allonger, et le driver finit par passer plus de temps à préparer le calcul que les machines à l'exécuter. `localCheckpoint` coupe court en remplaçant la recette par une simple lecture des données déjà calculées.
- **Le symptôme est trompeur.** Sur Amazon Movies, après environ huit tours de filtrage, le driver s'arrêtait sur une erreur de mémoire saturée. Or ce n'est pas en traitant les données que Spark manquait de mémoire, mais en *affichant* la recette devenue gigantesque. Cela ressemble à un problème de volume de données ; c'en est un de taille de plan d'exécution.
- **La leçon générale.** Dans une boucle Spark, il faut périodiquement couper l'historique des opérations. Notre boucle d'entraînement y échappe seulement parce que le modèle vit en dehors de Spark, ce qui est précisément le choix de la Section 3.4.

4.5 Environnement d'exécution des processus distants

Le notebook de l'Annexe B importe les modules en ajoutant `../src` à `sys.path`. Cela fonctionne sur le driver et échoue sur les workers :

```
1 src = os.path.dirname(os.path.abspath(__file__))
2 if src not in os.environ.get("PYTHONPATH", "").split(os.pathsep):
3     os.environ["PYTHONPATH"] = os.pathsep.join(
4         [src] + [p for p in os.environ.get("PYTHONPATH", "").split(os.pathsep) if p])
```

Listing 5: `spark_utils.get_spark` - trois lignes sans lesquelles rien ne fonctionne en dehors de `src/`.

Les processus de calcul lancés par Spark sont des processus Python distincts. Ils n'héritent pas des chemins d'import du programme principal : tout code envoyé sur un worker et qui importe nos modules échoue donc avec un `ModuleNotFoundError`. L'erreur est d'autant plus déroutante qu'elle survient à l'intérieur d'une tâche distante et remonte enfouie sous plusieurs traces de pile Java.

La correction tient en trois lignes : les processus héritent en revanche des variables d'environnement, il suffit donc d'y inscrire le chemin du code avant de démarrer Spark. Nos scripts ne rencontraient jamais le problème, car ils s'exécutent depuis le dossier `src/` ; le notebook, lancé depuis un autre dossier, l'a rencontré immédiatement. C'est typiquement le genre de bug qu'un environnement de développement unique dissimule jusqu'au jour où quelqu'un d'autre exécute le code.

4.6 Évaluer sans construire la matrice des scores

Pour évaluer le modèle, il faut classer l'article de test de chaque utilisateur parmi *tout* le catalogue. Calculer tous ces scores d'un coup demanderait 5,8 Go sur Amazon Movies. Les calculer un utilisateur à la fois en Python prendrait des minutes. Nous procédons par blocs de 256 utilisateurs, ce qui ramène le tout à une seule multiplication de matrices par bloc :

```
1 S = P[us] @ Q.T # (256, N) - un seul GEMM, pas 256 produits
   scalaires
2 for b, r in enumerate(buf):
3     S[b, np.asarray(r[1])] = -np.inf # masquer les items d'entraînement de cet
   utilisateur
4 st = S[np.arange(len(buf)), tis].copy()
5 rank = (S > st[:, None]).sum(1) + 1 # rang (base 1) de l'item verite-terrain
```

Listing 6: `evaluate._block_eval` - classement sur le catalogue entier, par blocs.

La taille du bloc plafonne la mémoire à 45 Mo au lieu de 5,8 Go, et l'évaluation est elle aussi distribuée : les utilisateurs sont répartis entre les cœurs comme le reste du calcul.

5 Analyse expérimentale

Chaque chiffre ci-dessous est produit par `src/experiments.py`, stocké en CSV brut dans `results/`, puis transformé en figure par `src/plots.py` et en tableau LaTeX par `report/tables.py`. Aucun chiffre de ce rapport n’est saisi à la main. Le matériel et les deux pièges de configuration sont décrits en Section 2.4 ; sauf mention contraire $K = 32$, $\lambda = 0,01$, $c_0 = 512$, $\alpha = 0,4$, graine 42, et 3 répétitions de 4 itérations dont la première est écartée comme échauffement. La ligne de commande exacte de chaque exécution figure dans `scripts/run_all.sh`.

Avertissement sur la comparabilité des mesures. Toute la campagne est une séquence de jobs à 8 cœurs sur un ordinateur portable, et un portable sous charge soutenue ne maintient pas une horloge constante. Nous avons mesuré la conséquence plutôt que de la balayer d’un revers de main : la même configuration ré-exécutée une heure plus tard, après un voisin plus lourd, est ressortie jusqu’à deux fois plus lente. Nous attribuons cela à une combinaison de limitation thermique (*throttling*) et de workers JVM/Python froids dans chaque `SparkSession` nouvellement créée, et nous n’avons pas cherché à séparer les deux effets. La règle pratique que nous avons suivie, et que le lecteur devrait appliquer en lisant les tableaux : **chaque conclusion ci-dessous est tirée d’un rapport calculé à l’intérieur d’un seul appel d’expérience** - une accélération, un facteur de croissance, un rapport entre méthodes - et jamais de secondes absolues comparées entre deux appels. Là où une expérience aurait sinon été exposée à cette dérive, elle a été ré-exécutée avec les répétitions entrelacées entre configurations et un temps de repos entre elles (`-interleave`, `-cooldown`).

5.1 Exactitude de la parallélisation

Rien d’autre dans cette section n’a de sens si le code distribué ne calcule pas eALS. Deux vérifications sont effectuées, dans cet ordre. D’abord, les règles de mise à jour elles-mêmes sont comparées à une évaluation littérale en $O(MNK)$ de l’Éq. (5) sur une matrice jouet, qui concorde à $3 \cdot 10^{-16}$ près (Section 3). Ensuite - le tableau ci-dessous - l’implémentation distribuée est comparée à la référence séquentielle. La Section 3 a argumenté que le découpage en blocs ne peut pas changer le résultat ; ici nous le vérifions. L’implémentation RDD est exécutée sur ml-100k avec la même graine, $K = 8$ et 5 itérations, pour 1, 2, 4 et 8 blocs, et comparée entrée par entrée à la référence NumPy séquentielle.

Table 4: Contrôle d’exactitude (`check_correctness.py`, ml-100k, $K = 8$, 5 itérations). « blocs » est le nombre de partitions Spark. L’objectif est l’Éq. (2) évaluée via l’Éq. (11).

Implémentation	blocs	$\max \Delta P $	$\max \Delta Q $	Objectif J	err. rel.
numpy-local	1	0.00e+00	0.00e+00	45890.818115	0.00e+00
spark-rdd	1	0.00e+00	0.00e+00	45890.818115	0.00e+00
spark-rdd	2	5.42e-14	1.28e-15	45890.818115	1.59e-16
spark-rdd	4	6.71e-14	1.53e-15	45890.818115	3.17e-16
spark-rdd	8	5.77e-14	1.28e-15	45890.818115	1.59e-16

Deux observations. Avec un seul bloc, l’exécution Spark est **identique au bit près** à la référence NumPy - la même arithmétique dans le même ordre. Avec 2, 4 ou 8 blocs, l’écart est de $7 \cdot 10^{-14}$, soit quelques unités du dernier chiffre d’un double : la seule chose qui change est l’ordre dans lequel les sommes partielles de S^p et S^q sont accumulées. L’objectif concorde à une erreur relative inférieure à 10^{-15} . C’est la forme empirique de l’affirmation de l’article selon laquelle eALS est trivialement parallélisable *sans perte d’approximation*, et cela signifie aussi que chaque mesure de temps ci-dessous mesure le même calcul.

5.2 Scalabilité

5.2.1 Scalabilité forte

Données fixes, ressources croissantes : `local[1]`, `local[2]`, `local[4]`, `local[8]`, avec un nombre de partitions maintenu proportionnel au nombre de cœurs pour que le travail par tâche reste constant.

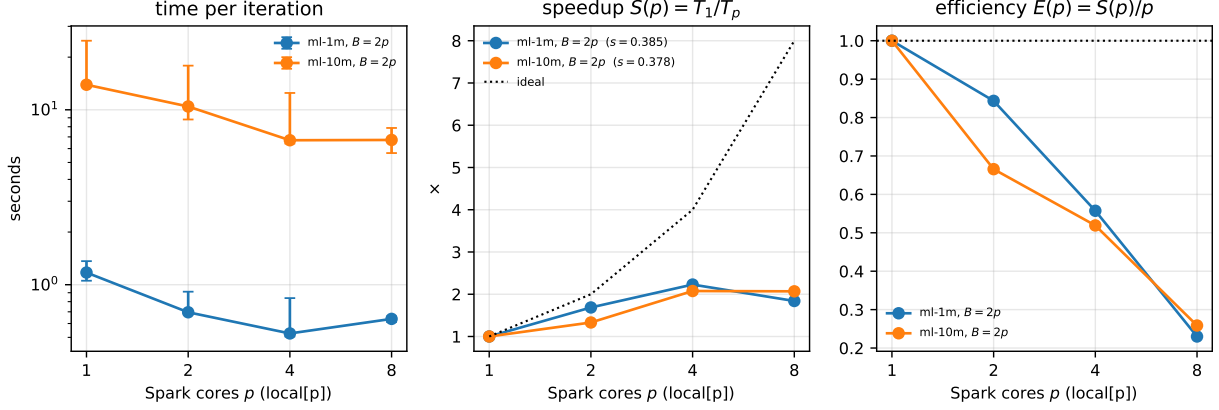


Figure 2: Scalabilité forte sur ml-1m (10⁶ interactions) et ml-10m (10⁷). Les barres d'erreur sont le min-max sur 9 échantillons par itération. Pointillés : l'accélération linéaire idéale ; tirets : la courbe d'Amdahl ajustée à chaque jeu de données.

Table 5: Scalabilité forte, $K = 32$, partitions = 2 × cœurs.

Cœurs p	médiane s/itér.	min-max	accélération $S(p)$	efficacité $E(p)$
<i>ml-1m</i> , $B = 2p$ - fitted Amdahl serial fraction $s = 0.385$				
1	1.174	1.054–1.364	1.00	100%
2	0.696	0.679–0.912	1.69	84%
4	0.526	0.506–0.838	2.23	56%
8	0.638	0.612–0.658	1.84	23%
<i>ml-10m</i> , $B = 2p$ - fitted Amdahl serial fraction $s = 0.378$				
1	13.908	13.655–24.822	1.00	100%
2	10.446	8.803–17.882	1.33	67%
4	6.698	6.412–12.480	2.08	52%
8	6.723	5.658–7.873	2.07	26%

Méthodologie. Le nombre de blocs croît avec le nombre de cœurs ($B = 2p$), de sorte que le travail par tâche reste constant - ce que l'on ferait en production. La contrepartie, qui compte pour la lecture des chiffres, est que la surcharge totale par tâche croît elle aussi avec p : à $p = 8$ l'exécution paie seize surcharges de bloc contre deux à $p = 1$, ce qui *sous-estime* l'accélération. Les répétitions sont *entrelacées* - la boucle externe est la répétition, la boucle interne le nombre de cœurs - afin que la lente dérive thermique d'un portable sous une campagne prolongée ne retombe pas entièrement sur la configuration exécutée en dernier. Nous avons constaté que cela comptait : dans une première exécution non entrelacée, le point à 8 cœurs a produit un unique échantillon de 19,7s contre une médiane de 8,8s.

Interprétation de l'ajustement d'Amdahl. En suivant Amdahl [1], nous ajustons $T(p) = a + b/p$ par moindres carrés et rapportons $s = a/(a + b)$, la fraction d'une itération à un cœur qui ne bénéficie pas de cœurs supplémentaires. Ajuster le modèle directement est plus robuste que de lire s sur le point à 8 cœurs, qui est exactement le point le plus exposé à la contention mémoire.

Résultats. L'exécution de référence - ml-10m, répétitions entrelacées - atteint $S(8) = 2.07\times$ sur un possible $8\times$, une efficacité de 26 %, avec une fraction série d'Amdahl ajustée de 0.378. Sur ml-1m,

où les données sont dix fois plus petites et où le coût fixe d’une itération en représente une part bien plus grande, $S(8) = 1.84\times$ et $s = 0.385$. Les deux courbes s’aplatissent entre 4 et 8 cœurs : le quatrième cœur rapporte encore quelque chose, le huitième pratiquement pas.

Origine de la fraction non parallélisable. Quatre choses ne se réduisent pas quand p croît :

- **La désérialisation des broadcasts**, payée une fois par worker Python et par broadcast, et donc *croissante* avec p . C’est le terme dominant, et il est invisible dans toute décomposition mesurée côté driver : `sc.broadcast` n’écrit le tableau qu’une fois, tandis que la moitié coûteuse - la désérialisation - se produit dans chaque worker Python et est imputée à la tâche, pas au driver.
- **La colle côté driver** : créer les broadcasts, disperser $2 \times B$ blocs dans P et Q , réduire les partiels de Gram.
- **La soumission des jobs et l’ordonnancement des tâches**, $2B$ tâches par itération.
- **La bande passante mémoire**, qui n’est pas du tout du travail série mais se comporte comme tel : le noyau est un flux de gathers et de sommes par segment sur des tableaux bien plus grands que le L2, et les huit cœurs d’un portable série M partagent un seul contrôleur mémoire. C’est pourquoi le plafond est une propriété de la machine plutôt que du code, et pourquoi un cluster de huit nœuds à un cœur paraîtrait meilleur qu’un seul nœud à huit cœurs.

5.2.2 Scalabilité en nombre de facteurs latents

C’est l’expérience qui décide si l’affirmation centrale de l’article survit au contact d’une implémentation réelle. Par itération, eALS coûte $O((M+N)K^2 + |\mathcal{R}|K)$ et ALS $O((M+N)K^3 + |\mathcal{R}|K^2)$. Lire cela comme « K^2 contre K^3 » est une erreur, et l’erreur se manifeste immédiatement dans les mesures. Les deux bornes sont la *somme* de deux termes, et lequel domine dépend de la forme des données :

$$(M+N)K^2 \geq 2|\mathcal{R}|K \iff K \geq K^* = \frac{2|\mathcal{R}|}{M+N}. \quad (12)$$

Sous K^* , eALS est essentiellement *linéaire* en K ; ce n’est qu’au-dessus que le terme quadratique prend le dessus. $K^* = 213$ sur ml-1m et $K^* = 34$ sur Amazon Movies, donc les deux jeux de données se situent de part et d’autre de la frontière intéressante et nous exécutons le balayage sur les deux. (Pour ALS, le même raisonnement donne un régime cubique au-dessus de $|\mathcal{R}|/(M+N)$, c.-à-d. la moitié de K^* .)

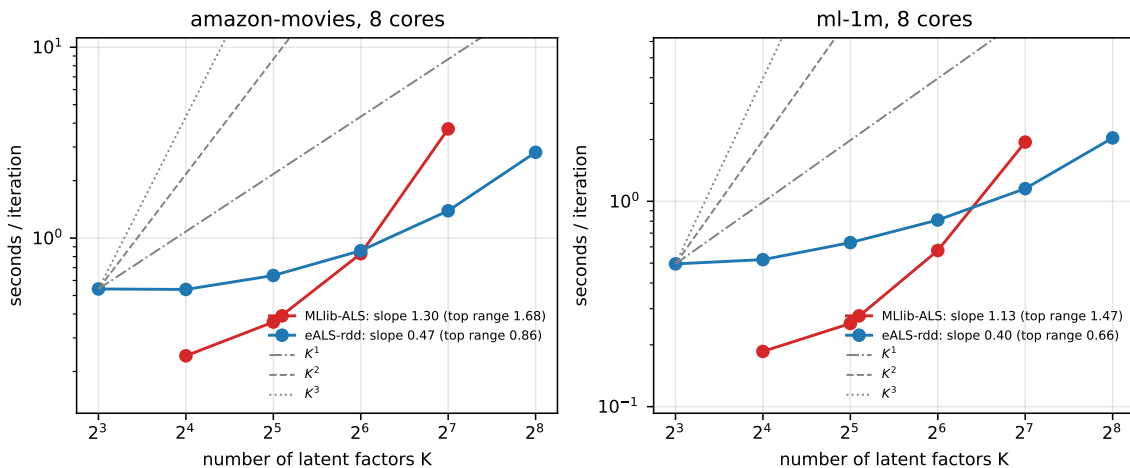


Figure 3: Coût d’une itération en fonction de K , 8 cœurs, log-log. Les repères gris sont K , K^2 et K^3 purs, ancrés au plus petit K . « plage haute » est la pente ajustée sur les deux dernières octaves seulement, où le coût constant de Spark ne compte plus.

Table 6: Secondes par itération en fonction de K , 8 cœurs. eALS est notre implémentation Spark RDD ; ALS est `pyspark.ml.recommendation.ALS(implicitPrefs=True)`, c.-à-d. Hu et al. [6], chronométré par son coût *marginal* $(T_{11} - T_1)/10$ afin d'exclure le coût fixe de construction de ses blocs. ALS n'a pas été exécuté au-delà de $K = 128$: une seule itération y coûte déjà des minutes.

K	eALS (s/itér.)	Mllib ALS (s/itér.)	rapport
<i>amazon-movies</i>			
8	0.54	-	-
16	0.54	0.24	0.4×
32	0.64	0.36	0.6×
64	0.86	0.83	1.0×
128	1.39	3.73	2.7×
256	2.82	-	-
<i>ml-1m</i>			
8	0.50	-	-
16	0.52	0.19	0.4×
32	0.63	0.25	0.4×
64	0.81	0.58	0.7×
128	1.15	1.94	1.7×
256	2.04	-	-

Résultats. L'affirmation la plus nette est un facteur de croissance sur une fenêtre fixe plutôt qu'un exposant ajusté. Sur **Amazon Movies**, entre $K = 32$ et $K = 128$, le coût d'une itération eALS est multiplié par 2.2 et celui d'une itération Mllib ALS par 10.3 - ALS croît près de cinq fois plus vite sur la même fenêtre, ce qui est l'affirmation qualitative de l'article. La conséquence en termes absolus est un point de croisement : à $K = 16$, Mllib ALS est 2.2× *plus rapide* que notre eALS, et à $K = 128$, eALS est 2.7× plus rapide. Sur **ml-1m**, la même fenêtre donne 1.8 contre 7.6 : le rapport entre les deux taux de croissance est essentiellement le même, sur un jeu de données dont la forme est complètement différente.

La forme théorique s'ajuste, et elle nous dit où se trouve réellement le croisement. Ajuster $t = a + bK + cK^2$ pour eALS et $t = a + bK^2 + cK^3$ pour ALS - les deux bornes de complexité, avec des constantes libres - reproduit les mesures presque exactement : $R^2 = 0.9996$ et 0.9999 sur Amazon Movies, 0.9995 et 1.0000 sur ml-1m. Les constantes additives sont 0.47s pour eALS contre 0.23s pour Mllib : notre implémentation Python paie environ le double du coût fixe par itération du code JVM natif, ce qui explique qu'ALS gagne à petit K .

Le chiffre intéressant est le rapport b/c , le K auquel le terme d'ordre supérieur dépasse le terme inférieur. Le décompte de flops prédit $K^* = 2|\mathcal{R}|/(M+N) = 34$ sur Amazon Movies ; le croisement *ajusté* est 315, neuf fois plus élevé. Sur ml-1m la prédiction est 213 et l'ajustement donne 1045.

Pourquoi le décompte de flops est faux d'un ordre de grandeur, et pourquoi c'est la chose la plus utile que dit cette expérience. Les deux termes de $O((M+N)K^2 + |\mathcal{R}|K)$ ont des constantes *par opération* très différentes dans toute implémentation réelle, et la borne de complexité les traite comme interchangeables. Dans notre noyau :

- le terme en K^2 est `Sq[:, f] @ PT`, un produit matrice-vecteur dense que BLAS exécute à plusieurs GFLOP/s hors cache ;
- le terme en $|\mathcal{R}|K$ est `QT[f][idx]` suivi de deux sommes par segment `np.bincount` - un gather aléatoire et un scatter aléatoire sur des tableaux de *nnz* éléments, limités par la latence mémoire, un ordre de grandeur plus lent par opération.

Un décompte de flops place donc K^* beaucoup trop bas, et l'exposant mesuré reste proche de 0.86 sur le haut de la plage où un décompte de flops pur prédirait presque 2. Ce n'est pas une erreur de l'analyse de l'article, qui est asymptotiquement correcte ; c'est l'écart ordinaire entre un décompte d'opérations et une hiérarchie mémoire, et cela a une conséquence pratique : **sur**

nos deux jeux de données, eALS se comporte, sur toute la plage de K qu'on peut se permettre d'entraîner, bien plus près d'un algorithme $O(|\mathcal{R}|K)$ que d'un algorithme $O((M+N)K^2)$ - ce qui est meilleur pour eALS que ne le suggère sa propre borne de complexité.

Réserve sur la méthode de référence. L'exposant de MLlib (1.68 sur la plage haute) est lui aussi bien en dessous de son 3 nominal, pour une raison du même type plus une autre : MLlib n'utilise pas une inversion de manuel. Elle accumule une équation normale et la factorise par Cholesky sur un stockage triangulaire compact, en code natif. L'article fait la même observation à propos de sa propre référence ALS en Java. La formulation honnête de cette expérience est donc « un algorithme quadratique en K écrit en Python contre un algorithme quasi-cubique bien optimisé écrit en code natif » - et le quadratique gagne quand même à partir de $K = 128$, par une marge qui croît.

5.3 Convergence et précision

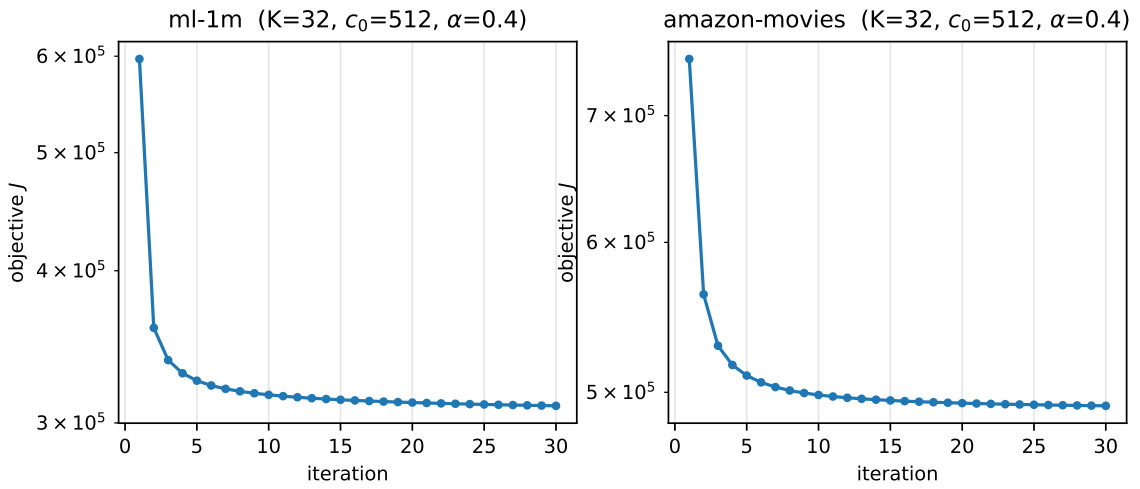


Figure 4: L'objectif (2), calculé via (11), à chaque itération ($K = 32$, $c_0 = 512$, $\alpha = 0,4$). La décroissance monotone est la preuve pratique que les règles de mise à jour sont les minimiseurs de coordonnées exacts.

Table 7: Convergence de l'objectif, $K = 32$, 8 cœurs. « itérations à 1 % » est la première itération dont l'objectif est à moins de 1 % de la valeur après 30 itérations.

Jeu de données	J après 1 itér.	J après 30	décroissance totale	itérations à 1 %	secondes
ml-1m	596903	309861	48.1%	17	18.8
amazon-movies	749932	491848	34.4%	12	14.1

L'objectif décroît de façon monotone à chaque itération sur les deux jeux de données, ce qui est le contrôle pratique que (9) et (10) sont réellement les minimiseurs de coordonnées exacts - un pas de descente de coordonnées même légèrement faux se manifesterait immédiatement ici par une remontée. La convergence est rapide dans le sens qui compte : 17 itérations sur ml-1m et 12 sur Amazon Movies amènent J à moins de 1 % de sa valeur après 30, et l'essentiel de la décroissance totale se produit dans les trois premières. Un budget de 20 itérations, utilisé partout ailleurs dans ce rapport, est donc largement au-delà du point où l'objectif cesse de bouger.

Trois constats, sur Amazon Movies, le jeu de données de l'article.

(i) eALS bat nettement l'ALS de MLlib en précision. HR@100 passe de 0.1171 à 0.1578, soit +35 %, avec NDCG@100 s'améliorant dans la même proportion. Les deux sont entraînés sur les mêmes données avec le même λ , le même K et le même nombre d'itérations. L'article rapporte le même ordre sur son propre instantané Amazon, et l'attribue à l'objectif sensible à la popularité plutôt qu'à l'optimiseur - ce que confirme le point suivant.

Table 8: Protocole hors-ligne : leave-one-out, classement contre le catalogue *entier*. $K = 32$, $\lambda = 0,01$, $c_0 = 512$, $\alpha = 0, 4, 8$ cœurs. « fit » est le temps d’entraînement (horloge murale) du modèle rapporté.

Méthode	HR@10	HR@100	NDCG@10	NDCG@100	fit (s)
<i>ml-1m</i>					
MostPopular	0.0230	0.1535	0.0114	0.0359	0.0
eALS	0.0593	0.3674	0.0281	0.0862	30.4
eALS-uniform(alpha=0)	0.0517	0.3387	0.0252	0.0796	31.7
MLlib-ALS	0.0535	0.3373	0.0255	0.0793	14.5
<i>amazon-movies</i>					
MostPopular	0.0097	0.0516	0.0046	0.0127	0.0
eALS	0.0291	0.1578	0.0142	0.0386	44.4
eALS-uniform(alpha=0)	0.0250	0.1434	0.0121	0.0345	41.8
MLlib-ALS	0.0216	0.1171	0.0106	0.0287	21.8

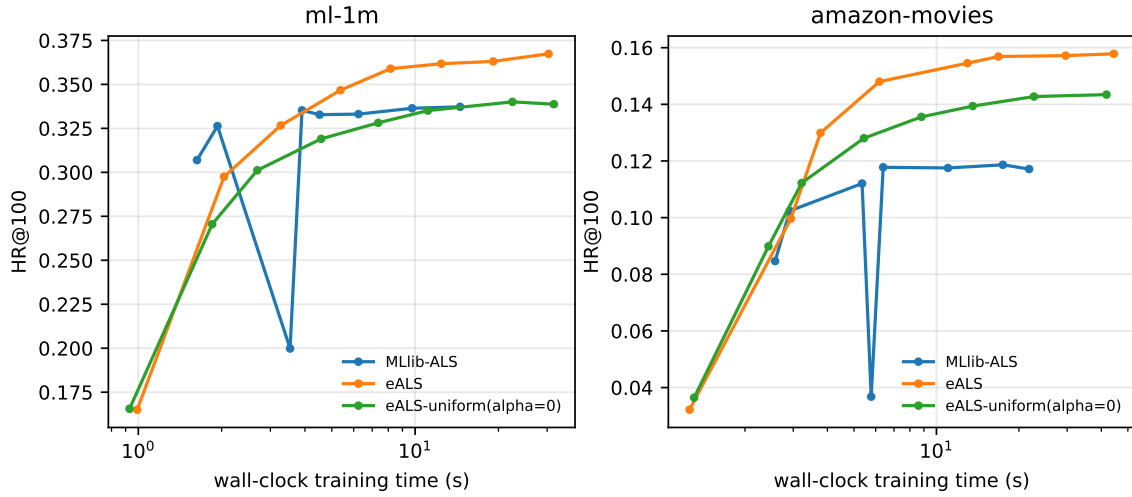


Figure 5: Précision en fonction du temps d’entraînement *réel* (horloge murale), pas du numéro d’itération. Chaque marqueur est une exécution d’entraînement complète de la durée donnée, évaluée sur l’ensemble retenu.

(ii) **La pondération sensible à la popularité vaut à elle seule 10.0 % de HR@100.** La seule différence entre les deux lignes eALS du Tableau 8 est $\alpha = 0,4$ contre $\alpha = 0$; tout le reste, y compris c_0 , la graine et le nombre d’itérations, est identique. La variante uniforme se situe entre MLlib ALS et eALS complet, exactement là où l’argument de l’article dit qu’elle devrait se trouver : une partie du gain vient d’une bonne optimisation de l’objectif sur l’ensemble des données, une partie d’une bonne pondération des données manquantes. Les deux modèles sont très au-dessus du plancher MostPopular ($3.1 \times$ son HR@100).

(iii) **Sur la métrique qui compte en pratique - la précision pour un budget de temps donné - eALS domine à chaque budget.** La Figure 5 est la comparaison honnête, car une comparaison par itération masquerait le fait qu’une itération ALS et une itération eALS ne représentent pas la même quantité de travail. eALS atteint la meilleure précision qu’ALS n’atteint jamais (HR@100 = 0.1186) après 3.8s d’entraînement, contre 17.5s pour ALS, puis continue de s’améliorer alors qu’ALS a plafonné : MLlib ALS est plat à partir de l’itération 8, à HR@100 $\approx 0,118$. Notez que ceci n’est *pas* une affirmation sur la vitesse brute - à $K = 32$ une itération ALS sur ces données est moins chère que la nôtre, car MLlib est du code JVM natif et nous payons la sérialisation Python. C’est une affirmation sur l’endroit où converge chaque algorithme.

6 Discussion : forces et faiblesses

6.1 Points forts

S1 - Quadratique, pas cubique, en K . C’est l’affirmation centrale de l’article, et la Section 5.2.2 la confirme dans la forme qui résiste à l’examen : sur la fenêtre $K = 32 \rightarrow 128$ sur Amazon Movies, une itération eALS croît de $2.2 \times$ et une itération ALS de $10.3 \times$. Comme la précision de la MF implicite continue de s’améliorer avec K , le régime où elle est la plus performante est précisément le régime où eALS gagne par la plus grande marge - et le croisement à $K = 128$ se situe en dessous du K que l’on veut utiliser.

S2 - Pas d’inversion de matrice, donc pas de problème de conditionnement. Chaque mise à jour de coordonnée est une division scalaire. ALS doit inverser $Q^\top W^u Q + \lambda I$ une fois par utilisateur ; à $K = 256$ avec un Q mal conditionné, c’est là que vivent les ennuis numériques, et c’est aussi pourquoi MLlib a besoin d’un chemin Cholesky avec un plancher de régularisation. eALS n’a pas un tel objet.

Une réserve que l’article ne soulève pas, et que nous pouvons préciser. Le dénominateur de (9) est $\sum_{i \in \mathcal{R}_u} (w_{ui} - c_i) q_{if}^2 + s_{ff}^q + \lambda$, dans lequel $s_{ff}^q = \sum_i c_i q_{if}^2 > 0$ et $\lambda > 0$, mais la première somme n’est *pas* de signe défini : rien dans le modèle n’interdit $c_i > w_{ui}$. Comme $c_i \approx c_0/N$ en moyenne, cela se produit sur les petits catalogues - sur ml-100k avec le $c_0 = 512$ de l’article et $N = 1\,152$, l’item le plus populaire atteint $c_i = 1,07 > 1 = w_{ui}$. Nous n’avons jamais observé d’objectif non monotone (Figure 4), car s_{ff}^q domine en pratique, mais la garantie est empirique plutôt que structurelle, et c’est une raison de faire varier c_0 avec N plutôt que de transposer une valeur d’un jeu de données à l’autre.

S3 - Un parallélisme exact. La Section 5.1 mesure un écart maximal de $7 \cdot 10^{-14}$ entre la référence NumPy séquentielle et les exécutions Spark à 1, 2, 4 et 8 blocs, et l’exécution à un bloc est identique au bit près. Rien n’a besoin d’être ajusté, aucune obsolescence (*staleness*) n’a besoin d’être tolérée, aucun taux d’apprentissage n’a besoin d’être re-recherché quand le nombre de workers change - ce que la SGD distribuée ne peut précisément pas promettre. Opérationnellement cela vaut plus qu’il n’y paraît : cela signifie que le nombre de partitions est un pur bouton de performance, et qu’une expérience de scalabilité ne peut pas changer silencieusement le modèle qu’elle mesure.

S4 - Pas de taux d’apprentissage. Chaque mise à jour est le minimiseur exact de l’objectif le long d’une coordonnée. Comparé à RCD [3], qui a besoin d’une recherche linéaire à chaque pas, ou à

BPR [10], dont la précision est visiblement fonction de la taille du pas, cela retire une dimension entière du budget d’ajustement. Notre propre expérience le confirme : aucune configuration que nous ayons exécutée n’a jamais divergé, et l’objectif décroît de façon monotone à chaque itération sur les deux jeux de données (Figure 4).

6.2 Points faibles

W1 - Le balayage de coordonnées est intrinsèquement séquentiel à l’intérieur d’une ligne. La boucle en K est du Gauss–Seidel : $p_{u,f+1}$ doit voir le p_{uf} mis à jour. Le parallélisme est donc limité à $\min(M, B)$ dans l’étape utilisateur et $\min(N, B)$ dans l’étape item, et le travail par bloc contient toujours une boucle Python de longueur K . C’est cette boucle qui a obligé l’implémentation à être vectorisée *entre* utilisateurs : c’est le seul axe restant. Dans un contexte à peu d’utilisateurs et à K énorme - l’opposé d’un système de recommandation - eALS se paralléliserait mal.

W2 - Diffuser Q ne passe pas à l’échelle des catalogues réels. La conception RDD exige P et Q sur le driver et une copie désérialisée de chacun par worker Python. À $N = 10^6$ items et $K = 128$ cela fait 1 Go par worker. C’est la limite structurelle de la conception retenue en Section 2, et elle est assumée : l’alternative - faire transiter les facteurs par des jointures avec shuffle - déplace $O(|\mathcal{R}|K)$ au lieu de $O((M + N)K)$, soit un facteur ≈ 300 sur ml-1m, et se paie à chaque itération. Il n’y a pas de repas gratuit ici : à une certaine taille de catalogue, la conception fondée sur les jointures gagne simplement parce que la conception broadcast cesse de tenir en mémoire, et la bonne réponse industrielle n’est probablement ni l’une ni l’autre, mais le schéma à partitionnement par blocs de MLlib, où chaque ligne de facteur n’est envoyée qu’aux blocs qui en ont besoin.

W3 - Le schéma de pondération exige une recherche bidimensionnelle coûteuse. c_0 et α interagissent, les propres optima de l’article diffèrent entre ses deux jeux de données ($c_0 = 512, \alpha = 0, 4$ sur Yelp ; $c_0 = 64, \alpha = 0, 5$ sur Amazon), et chaque point de la grille est un entraînement complet - une recherche bidimensionnelle sérieuse représente donc des dizaines d’entraînements. Nous avons repris les valeurs de l’article plutôt que de les rechercher, ce qui est précisément la raison pour laquelle nos chiffres de précision ne doivent pas être lus comme un optimum. Pire, la sensibilité n’est pas symétrique : un c_0 trop petit détruit la précision, un c_0 trop grand la dégrade doucement, donc une grille grossière peut facilement tomber sur un plateau et déclarer victoire.

W4 - Un protocole d’évaluation indulgent, en particulier envers la pondération par popularité. Trois problèmes distincts :

- *Le leave-one-out par horodatage n’est pas un découpage réaliste.* Il laisse fuir le futur : l’ensemble d’entraînement de l’utilisateur u contient des interactions survenues après l’interaction de test de l’utilisateur v . L’article le dit lui-même et ajoute pour cette raison un protocole en ligne fondé sur une coupure chronologique globale, que nous n’avons pas reproduit ; tous nos chiffres de précision héritent donc de ce biais.
- *Des négatifs échantillonnés rendraient les chiffres incomparables.* Nous classons contre le catalogue entier précisément parce qu’échantillonner 100 négatifs [7] est connu pour changer non seulement les valeurs mais l’ordre relatif des systèmes comparés (Krichene et Rendle, KDD 2020). Cela coûte $O(MNK)$ par évaluation et c’est la principale raison pour laquelle notre évaluation est distribuée.
- *HR et NDCG ne mesurent rien sur la diversité ou la couverture du catalogue.* C’est la critique que nous jugeons la plus importante, car elle pointe vers la contribution même de l’article. Pondérer les données manquantes par la popularité indique au modèle qu’un manque sur un blockbuster est une forte preuve de désintérêt - mais le même mécanisme pousse les items impopulaires vers un score prédit bas pour tout le monde, car il est bon marché de se tromper sur eux. Une métrique qui demande seulement « l’item retenu était-il dans le top 100 » ne

peut pas voir la concentration qui en résulte, et les items retenus sont eux-mêmes distribués selon la popularité, donc la métrique est biaisée positivement en faveur de tout ce qui amplifie la popularité. Nous mesurons un vrai gain à partir de $\alpha > 0$ (Section 5.3), et nous croyons qu'il est réel ; nous croyons aussi qu'une évaluation rapportant la couverture dans l'espace des items montrerait qu'une partie de ce gain est un biais de popularité plutôt qu'une meilleure modélisation des préférences. Mesurer cela est l'extension la plus utile de ce travail.

W5 - eALS optimise une perte de régression, pas une perte de classement. L'objectif est l'erreur quadratique sur une matrice 0/1. C'est un bon objectif de *rappel*, ce que montrent exactement nos résultats - eALS est le plus fort en HR@100 - et un objectif de *précision* médiocre. BPR, qui optimise directement l'ordre par paires, est le concurrent naturel aux petites coupures ; cette asymétrie entre rappel et précision est un résultat classique de la littérature sur la recommandation top- N [2]. Nous n'avons pas implémenté BPR (voir plus bas) et ne pouvons donc pas le quantifier sur nos données ; l'article rapporte le même schéma.

Un point connexe et plus inconfortable : Rendle et al. ont revisité iALS - la référence même à laquelle eALS est comparé - sur des benchmarks standards et ont trouvé qu'une fois sa régularisation et son poids sur les données non observées correctement ajustés, il rivalise avec des méthodes bien plus récentes. Notre propre référence ALS est celle de MLlib, avec le $\lambda = 0,01$ de l'article et l' α par défaut de MLlib ; nous ne l'avons *pas* ajustée. Une partie de l'écart de 35 % que nous mesurons est donc attribuable à un effort d'ajustement inégal, et une lecture honnête du Tableau 8 devrait le traiter comme une borne supérieure de l'avantage d'eALS sur un iALS bien ajusté.

6.3 Limites de validité des mesures

Nous les listons car la plupart sont structurelles et ne peuvent pas être corrigées sur un portable.

1. **local[p] n'est pas un cluster.** Il n'y a pas de réseau, pas de sérialisation entre machines, pas de tâche à la traîne due à un nœud lent, pas de tolérance aux pannes exercée. Nos chiffres *surestiment* donc la performance de la conception RDD sur un vrai cluster, où diffuser Q coûte de la bande passante réseau et non une copie mémoire, et *sous-estime* d'autant la valeur de l'alternative co-partitionnée écartée en Section 2.
2. **La bande passante mémoire partagée est la vraie limite, pas le nombre de cœurs.** Le noyau est une séquence de gathers et de sommes par segment sur des tableaux bien plus grands que le L2. Passer de 1 à 8 cœurs multiplie la demande sur un seul contrôleur mémoire, ce qui explique une grande partie de l'écart entre l'accélération mesurée $2.07\times$ et l'idéal $8\times$. Un cluster de 8 machines à un cœur paraîtrait meilleur qu'une seule machine à 8 cœurs ici.
3. **La dérive thermique est mesurable, et nous l'avons attrapée.** Deux exécutions du même balayage en K sur ml-1m, à une heure d'intervalle dans une campagne continue, différeraient jusqu'à un facteur deux sur les configurations les plus lourdes : la seconde exécution suivait une mesure ALS bien plus lourde et la machine était chaude. Un portable sous charge soutenue à 8 cœurs ne maintient pas une horloge constante. Nos parades sont méthodologiques plutôt que physiques - répétitions entrelacées entre configurations plutôt que regroupées, et un temps de repos entre configurations (`-interleave`, `-cooldown`) - et les figures montrent des barres min-max sur 9 échantillons par itération afin que l'étalement résiduel soit visible. Sur une machine dédiée à horloge fixe, ces précautions seraient inutiles ; sur celle-ci, elles font la différence entre une mesure et une anecdote.
4. **Deux familles de jeux de données, un seul domaine chacune.** Amazon Movies et MovieLens sont tous deux de nature cinématographique, tous deux modérément denses après filtrage 10-core. Rien ici ne dit comment eALS se comporte sur un catalogue de 10^7 items avec une moyenne de deux interactions chacun.
5. **Le notebook est le seul test de bout en bout.** Il n'y a pas de suite de tests unitaires ; l'exactitude est garantie par `check_correctness.py` (qui compare bien à une référence par

force brute) et par le fait que le notebook s'exécute jusqu'au bout. Cela a suffi à attraper de vrais bugs - notamment un `PYTHONPATH` manquant sur les workers - mais ce n'est pas un substitut à des tests.

6. **Tout ce qui figurait dans l'énoncé n'a pas été mesuré.** Nous n'avons pas implémenté BPR ni RCD : les deux sont des références pertinentes dans l'article, mais chacune est un projet en soi et le budget est allé à l'étude de scalabilité que l'énoncé pondère le plus. Nous le disons plutôt que de rapporter des chiffres que nous n'avons pas produits. Nous n'avons pas non plus utilisé Yelp, pour la raison donnée en Section 2.3.

7 Conclusion et travaux futurs

Nous avons implémenté eALS [5] en PySpark - une version RDD fondée sur des broadcasts, adossée à une référence NumPy séquentielle - et vérifié que les deux produisent le même modèle à $7 \cdot 10^{-14}$ près, quel que soit le nombre de blocs, ce qui est le pendant empirique de l'affirmation de l'article selon laquelle eALS est trivialement parallélisable *sans approximation*. En plus de cela, nous avons implémenté l'évaluation leave-one-out sur catalogue entier et une campagne expérimentale dont la sortie brute se trouve dans `results/*.csv`.

Les résultats que nous considérons comme la substance de ce rapport :

1. L'affirmation de complexité tient, dans le régime pour lequel elle est énoncée. Sur Amazon Movies, le coût d'une itération entre $K = 32$ et $K = 128$ croît de $2.2\times$ pour eALS et $10.3\times$ pour l'ALS implicite de MLlib, sur les mêmes données, la même machine et le même nombre de cœurs. eALS passe de $2.2\times$ plus lent à $K = 16$ à $2.7\times$ plus rapide à $K = 128$. Ajuster les deux formes de complexité avec des constantes libres reproduit les mesures à $R^2 > 0,999$, mais place le croisement entre le terme linéaire et le terme quadratique d'eALS à $K \approx 315$ plutôt qu'au 34 prédit par le décompte de flops : le terme creux est limité par la mémoire et le terme dense relève de BLAS, et un décompte de flops ne peut pas voir la différence.
2. La scalabilité forte est réelle mais modeste : $2.07\times$ sur 8 cœurs pour ml-10m, cohérent avec une fraction série d'Amdahl de 0.378. La limite n'est pas l'algorithme - le parallélisme est exact et le travail se découpe proprement - mais la bande passante mémoire partagée, la désérialisation des broadcasts dans chaque worker Python, et la surcharge Python par tâche.
3. La pondération sensible à la popularité fonctionne : à $\alpha = 0,4$, HR@100 passe de 0.1434 à 0.1578 sur Amazon Movies, soit 10.0% de mieux que le réglage uniforme $\alpha = 0$, toutes choses égales par ailleurs.
4. Sur la métrique qui compte en pratique - la précision pour un budget de temps donné - eALS atteint en 3.8s la meilleure précision que l'ALS de MLlib n'atteint jamais (HR@100 = 0.1186 après 17.5s), puis continue de s'améliorer.

Travaux futurs. Quatre directions, dans l'ordre où nous les prendrions. (i) *Métriques de couverture et de biais de popularité.* L'ajout le plus utile, de loin : mesurer la couverture du catalogue et la concentration de Gini en fonction de α , et voir quelle part du gain de HR relève réellement d'une meilleure modélisation des préférences. (ii) *Une implémentation co-partitionnée.* Remplacer le broadcast de Q par un échange par blocs partitionnés où chaque ligne de facteur n'est envoyée qu'aux blocs qui la référencent - le schéma qu'utilise l'ALS de MLlib - et ré-exécuter l'étude de scalabilité sur un vrai cluster, où le coût du broadcast cesse d'être une copie mémoire. (iii) *Références BPR et RCD,* pour tester l'asymétrie rappel/précision que nous ne pouvons actuellement que citer depuis l'article. (iv) *Float32 et une boucle interne Numba ou Cython.* Le noyau est limité par la mémoire ; diviser par deux la largeur de chaque gather devrait rapporter près d'un facteur deux, et diviserait aussi le broadcast par deux.

Déclaration sur l’usage d’outils d’assistance

La rédaction de ce rapport et l’écriture du code ont bénéficié de l’assistance d’un modèle de langage. Les mesures rapportées proviennent de nos propres exécutions sur la machine décrite en Section 2.4 ; les fichiers CSV bruts sont versionnés dans `results/` et l’ensemble des figures, tableaux et chiffres du rapport en est regénéré par `scripts/build_report.sh`.

References

- [1] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*, pages 483–485, 1967.
- [2] Paolo Cremonesi, Yehuda Koren, and Roberto Turrin. Performance of recommender algorithms on top-n recommendation tasks. In *Proceedings of the 4th ACM Conference on Recommender Systems (RecSys ’10)*, pages 39–46, 2010.
- [3] Robin Devooght, Nicolas Kourtellis, and Amin Mantrach. Dynamic matrix factorization with priors on unknown values. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD ’15)*, pages 189–198, 2015.
- [4] F. Maxwell Harper and Joseph A. Konstan. The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):19:1–19:19, 2015.
- [5] Xiangnan He, Hanwang Zhang, Min-Yen Kan, and Tat-Seng Chua. Fast matrix factorization for online recommendation with implicit feedback. In *Proceedings of the 39th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR ’16)*, pages 549–558, 2016.
- [6] Yifan Hu, Yehuda Koren, and Chris Volinsky. Collaborative filtering for implicit feedback datasets. In *Eighth IEEE International Conference on Data Mining (ICDM ’08)*, pages 263–272, 2008.
- [7] Walid Krichene and Steffen Rendle. On sampled metrics for item recommendation. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD ’20)*, pages 1748–1757, 2020.
- [8] Julian McAuley, Christopher Targett, Qinfeng Shi, and Anton van den Hengel. Image-based recommendations on styles and substitutes. In *Proceedings of the 38th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR ’15)*, pages 43–52, 2015.
- [9] Xiangrui Meng, Joseph Bradley, Burak Yavuz, Evan Sparks, Shivaram Venkataraman, Davies Liu, Jeremy Freeman, D.B. Tsai, Manish Amde, Sean Owen, Doris Xin, Reynold Xin, Michael J. Franklin, Reza Zadeh, Matei Zaharia, and Ameet Talwalkar. MLlib: Machine learning in apache spark. *Journal of Machine Learning Research*, 17(34):1–7, 2016.
- [10] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. BPR: Bayesian personalized ranking from implicit feedback. In *Proceedings of the 25th Conference on Uncertainty in Artificial Intelligence (UAI ’09)*, pages 452–461, 2009.
- [11] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI ’12)*, pages 15–28, 2012.

A Code source complet

Tout est également disponible sur <https://github.com/tomasnp/eals-spark>. Les listings ci-dessous sont les fichiers tels quels (verbatim, en anglais dans le code source).

A.1 src/spark_utils.py - session Spark

```
1 """Single place where the SparkSession is built, so every script runs with the
2 same configuration and the scalability experiments only differ by 'cores'."""
3 import os
4 import shutil
5 from pyspark.sql import SparkSession
6
7 SPARK_TMP = os.environ.get("EALS_SPARK_TMP", "/tmp/eals-spark-tmp")
8
9
10 def get_spark(app="eals", cores=8, driver_mem="10g", shuffle_partitions=None, quiet=True)
11 :
12     os.makedirs(SPARK_TMP, exist_ok=True)
13     # A module the *driver* can import is not automatically importable in the Python
14     # workers: they are separate processes and do not inherit sys.path. Every closure
15     # that references eals_local / eals_rdd would fail with ModuleNotFoundError. The
16     # workers do inherit the environment, so PYTHONPATH is the fix -- and it must be set
17     # before the JVM is launched, i.e. before getOrCreate().
18     src = os.path.dirname(os.path.abspath(__file__))
19     if src not in os.environ.get("PYTHONPATH", "").split(os.pathsep):
20         os.environ["PYTHONPATH"] = os.pathsep.join(
21             [src] + [p for p in os.environ.get("PYTHONPATH", "").split(os.pathsep) if p])
22     b = (
23         SparkSession.builder.master(f"local[{cores}]")
24         .appName(app)
25         # local[k] runs everything in the driver JVM, so driver memory is *the*
26         # memory knob: the broadcast copies of P and Q live here.
27         .config("spark.driver.memory", driver_mem)
28         .config("spark.driver.maxResultSize", "4g")
29         .config("spark.local.dir", SPARK_TMP)
30         .config("spark.ui.enabled", "false")
31         .config("spark.sql.shuffle.partitions", shuffle_partitions or (cores * 2))
32         # only used to move a training split to pandas (toPandas) for the NumPy
33         # reference and the evaluation.
34         .config("spark.sql.execution.arrow.pyspark.enabled", "true")
35     )
36     s = b.getOrCreate()
37     if quiet:
38         s.sparkContext.setLogLevel("ERROR")
39     return s
40
41 def clear_tmp():
42     shutil.rmtree(SPARK_TMP, ignore_errors=True)
```

A.2 src/data.py - chargement, filtrage 10-core, découpage

```
1 """Dataset loading, k-core filtering, dense re-indexing and the offline
2 leave-one-out split used by the paper.
3
4 All of it runs in Spark DataFrames; the output is parquet so that every
5 experiment starts from exactly the same materialised training set.
6 """
7 import argparse
8 import json
9 import os
10
11 from pyspark.sql import Window
12 from pyspark.sql import functions as F
13
14 from spark_utils import get_spark
15
16 RAW = os.environ.get("EALS_RAW", os.path.expanduser("~/eals-spark/data/raw"))
17 PROC = os.environ.get("EALS_PROC", os.path.expanduser("~/eals-spark/data/processed"))
```

```

18
19 DATASETS = ("ml-100k", "ml-1m", "ml-10m", "amazon-movies")
20
21
22 def load_raw(spark, name):
23     """-> DataFrame(user string, item string, ts long). Ratings are dropped:
24     implicit feedback only keeps the *existence* of an interaction."""
25     if name == "ml-100k":
26         p = f"{RAW}/ml-100k/u.data"
27         df = spark.read.csv(p, sep="\t", schema="user STRING, item STRING, rating DOUBLE,
28                             ts LONG")
29     elif name in ("ml-1m", "ml-10m"):
30         p = f"{RAW}/ml-1m/ratings.dat" if name == "ml-1m" else f"{RAW}/ml-10m100K/ratings
31         .dat"
32         df = (
33             spark.read.text(p)
34             .select(F.split("value", ":").alias("a"))
35             .select(F.col("a")[0].alias("user"), F.col("a")[1].alias("item"),
36                     F.col("a")[3].cast("long").alias("ts"))
37         )
38     elif name == "amazon-movies":
39         p = f"{RAW}/ratings_Movies_and_TV.csv"
40         df = spark.read.csv(p, schema="user STRING, item STRING, rating DOUBLE, ts LONG")
41     else:
42         raise ValueError(name)
43     return df.select("user", "item", "ts").dropna()
44
45 def kcore(df, k):
46     """Iterated k-core: dropping sparse users can push items below k and vice
47     versa, so both filters are re-applied until the interaction count is
48     stable. The k-core of a bipartite graph is unique, so the fixed point does
49     not depend on the order in which the two sides are pruned."""
50     df = df.cache()
51     prev = -1
52     while True:
53         n = df.count()
54         if n == prev:
55             return df
56         prev = n
57         ok_u = df.groupBy("user").count().filter(F.col("count") >= k).select("user")
58         ok_i = df.groupBy("item").count().filter(F.col("count") >= k).select("item")
59         nxt = df.join(ok_u, "user").join(ok_i, "item").select("user", "item", "ts")
60         # localCheckpoint, not cache: cache() keeps the logical plan alive, so
61         # after ~8 rounds the plan is deep enough that Spark itself dies with an
62         # OutOfMemoryError while *printing* it (TreeNode.generateTreeString).
63         # Checkpointing truncates the lineage to a plain scan of the materialised
64         # rows. This is the iterative-Spark trap, and it bites in data prep too.
65         nxt = nxt.localCheckpoint(eager=True)
66         df.unpersist()
67         df = nxt
68
69 def reindex(df):
70     """Dense 0..M-1 / 0..N-1 ids: the eALS kernels index NumPy arrays directly,
71     so ids must be contiguous. Ordering by descending frequency also puts the
72     heavy users first, which slightly balances the hash partitions."""
73     um = (df.groupBy("user").count().orderBy(F.desc("count"), "user")
74           .withColumn("u", F.row_number().over(Window.orderBy(F.desc("count"), "user"))
75               - 1)
76           .select("user", "u"))
77     im = (df.groupBy("item").count().orderBy(F.desc("count"), "item")
78           .withColumn("i", F.row_number().over(Window.orderBy(F.desc("count"), "item"))
79               - 1)
80           .select("item", "i"))
81     out = df.join(um, "user").join(im, "item").select("u", "i", "ts")
82     return out, um, im
83
84 def leave_one_out(df):
85     """Offline protocol: the chronologically last interaction of every user is
86     held out. Amazon timestamps are day-granular so ties are frequent; the item
87     id is used as a deterministic tie-break."""
88     w = Window.partitionBy("u").orderBy(F.desc("ts"), F.desc("i"))
89     r = df.withColumn("rk", F.row_number().over(w))

```



```

89     return r.filter("rk > 1").drop("rk"), r.filter("rk = 1").drop("rk")
90
91
92 def prepare(spark, name, k=10, out_dir=None, sample_frac=None, seed=42):
93     out_dir = out_dir or f"{PROC}/{name}"
94     df = load_raw(spark, name).dropDuplicates(["user", "item"])
95     if sample_frac:
96         # Sample *users*, not interactions: sampling interactions would destroy
97         # the k-core property and shift the popularity distribution.
98         users = df.select("user").distinct().sample(sample_frac, seed=seed)
99         df = df.join(users, "user")
100    df = kcore(df, k)
101    df, um, im = reindex(df)
102    df = df.cache()
103    tr, te = leave_one_out(df)
104
105    stats = dict(dataset=name, kcore=k, sample_frac=sample_frac,
106                users=df.select("u").distinct().count(),
107                items=df.select("i").distinct().count(),
108                interactions=df.count())
109    stats["sparsity"] = 1 - stats["interactions"] / (stats["users"] * stats["items"])
110    stats["train"], stats["test"] = tr.count(), te.count()
111
112    for sub, d in (("train", tr), ("test", te)):
113        d.write.mode("overwrite").parquet(f"{out_dir}/{sub}")
114    os.makedirs(out_dir, exist_ok=True)
115    json.dump(stats, open(f"{out_dir}/stats.json", "w"), indent=2)
116    return stats
117
118
119 def load_split(spark, name, sub="train", proc=None):
120     return spark.read.parquet(f"{proc or PROC}/{name}/{sub}")
121
122
123 if __name__ == "__main__":
124     ap = argparse.ArgumentParser()
125     ap.add_argument("--dataset", default="ml-1m", choices=DATASETS)
126     ap.add_argument("--kcore", type=int, default=10)
127     ap.add_argument("--cores", type=int, default=8)
128     ap.add_argument("--sample-frac", type=float, default=None)
129     ap.add_argument("--out", default=None)
130     a = ap.parse_args()
131     sp = get_spark("prepare", cores=a.cores)
132     print(json.dumps(prepare(sp, a.dataset, a.kcore, a.out, a.sample_frac), indent=2))
133     sp.stop()

```

A.3 src/eals_local.py - référence NumPy et noyaux partagés

```

1  """Single-machine NumPy eALS (He et al., SIGIR'16).
2
3  This module holds the two element-wise update kernels. 'eals_rdd.py' imports
4  them unchanged and only differs in *who* calls them and on which slice of the
5  data -- so any accuracy difference between the local and the distributed run can
6  only come from the scheduling, never from the math.
7
8  Notation follows the paper: P (M x K) users, Q (N x K) items, rui = 1 and
9  wui = 1 on observed entries, wui = ci on missing ones.
10 Factors are stored *transposed* (K x M / K x N) because the inner loop gathers
11 one whole coordinate f at a time; a row of the transposed matrix is contiguous.
12 """
13 import numpy as np
14
15
16 def item_confidence(item_counts, c0=512.0, alpha=0.4):
17     """Eq. (8): ci = c0 * fialpha / sumj fjalpha, fi = |Ri| / sumj |Rj|."""
18     f = item_counts / max(item_counts.sum(), 1.0)
19     fa = f ** alpha
20     return c0 * fa / fa.sum()
21
22
23 def sort_by_row(rows, cols):
24     """Sorting the entries by row turns the per-coordinate gather on the row
25     factors into a sequential scan instead of a random one."""

```

```

26 o = np.argsort(rows, kind="stable")
27 return rows[o].astype(np.int64), cols[o].astype(np.int64)
28
29
30 def _predict(FT, GT, rowid, idx):
31     """r_hat on the observed entries only, without materialising an (nnz, K)
32     gather: K passes of O(nnz) instead of one pass of O(nnz*K) memory."""
33     out = np.zeros(rowid.shape[0])
34     for f in range(FT.shape[0]):
35         out += FT[f][rowid] * GT[f][idx]
36     return out
37
38
39 def update_users(PT, QT, idx, rowid, cvec, Sq, lam, w=1.0, rhat=None):
40     """Eq. (12) applied to every user of a block, one coordinate f at a time.
41
42     Users are independent given (Q, Sq), so the whole block is updated with
43     vector operations: the Gauss-Seidel sweep stays sequential over f, but is
44     simultaneous over the users. Cost: O(nnz*K + m*K^2).
45     """
46     K, m = PT.shape
47     c_e = cvec[idx]
48     wmc = w - c_e # (w_u_i - c_i)
49     wr = w * 1.0 # w_u_i * r_u_i, r_u_i = 1
50     if rhat is None:
51         rhat = _predict(PT, QT, rowid, idx)
52     for f in range(K):
53         qf = QT[f][idx]
54         rhat -= PT[f][rowid] * qf # r_hat^f = r_hat - p_u f q_i f
55         num = np.bincount(rowid, (wr - wmc * rhat) * qf, minlength=m)
56         den = np.bincount(rowid, wmc * qf * qf, minlength=m)
57         num -= Sq[:, f] @ PT - PT[f] * Sq[f, f] # - sum_{k!=f} p_u k s^q_k f
58         den += Sq[f, f] + lam
59         PT[f] = num / den
60         rhat += PT[f][rowid] * qf
61     return rhat
62
63
64 def update_items(QT_blk, PT, idx, rowid, c_blk, Sp, lam, w=1.0, rhat=None):
65     """Eq. (13). Same structure as update_users, but the dense term and the
66     denominator are scaled by c_i (which is an item property)."""
67     K, n = QT_blk.shape
68     c_e = c_blk[rowid]
69     wmc = w - c_e
70     wr = w * 1.0
71     if rhat is None:
72         rhat = _predict(QT_blk, PT, rowid, idx)
73     for f in range(K):
74         pf = PT[f][idx]
75         rhat -= QT_blk[f][rowid] * pf
76         num = np.bincount(rowid, (wr - wmc * rhat) * pf, minlength=n)
77         den = np.bincount(rowid, wmc * pf * pf, minlength=n)
78         num -= c_blk * (Sp[:, f] @ QT_blk - QT_blk[f] * Sp[f, f])
79         den += c_blk * Sp[f, f] + lam
80         QT_blk[f] = num / den
81         rhat += QT_blk[f][rowid] * pf
82     return rhat
83
84
85 def objective(PT, QT, u_idx, i_idx, cvec, lam, w=1.0):
86     """Eq. (7) evaluated through Eq. (14) in O(|R| + M K^2) instead of O(MNK):
87     the missing-data term becomes sum_u p_u u^T S^q p_u - sum_{(u,i) in R} c_i r_u i^2."""
88     rhat = _predict(PT, QT, u_idx, i_idx)
89     Sq = (QT * cvec) @ QT.T
90     obs = float((w * (1.0 - rhat) ** 2).sum())
91     miss = float((PT * (Sq @ PT)).sum() - (cvec[i_idx] * rhat ** 2).sum())
92     reg = lam * (float((PT ** 2).sum()) + float((QT ** 2).sum()))
93     return obs + miss + reg
94
95
96 def fit(u_idx, i_idx, M, N, K=32, lam=0.01, c0=512.0, alpha=0.4, iters=20,
97         seed=42, init_std=0.01, w=1.0, trace=None):
98     """Algorithm 1, sequential reference implementation."""
99     rng = np.random.default_rng(seed)
100     PT = rng.normal(0, init_std, (K, M))

```

```

101 QT = rng.normal(0, init_std, (K, N))
102 cvec = item_confidence(np.bincount(i_idx, minlength=N).astype(float), c0, alpha)
103
104 urow, uidx = sort_by_row(u_idx, i_idx)
105 irow, iidx = sort_by_row(i_idx, u_idx)
106
107 for it in range(iters):
108     Sq = (QT * cvec) @ QT.T # Algorithm 1, line 4
109     update_users(P, QT, uidx, urow, cvec, Sq, lam, w)
110     Sp = P @ P.T # Algorithm 1, line 12
111     update_items(QT, P, iidx, irow, cvec, Sp, lam, w)
112     if trace is not None:
113         trace.append(objective(P, QT, u_idx, i_idx, cvec, lam, w))
114 return P.T.copy(), QT.T.copy(), cvec
115
116
117 if __name__ == "__main__":
118     # Runnable without Spark at all: the reference implementation reads the
119     # parquet produced by data.py straight into pandas.
120     import argparse
121     import glob
122     import os
123     import time
124
125     import pandas as pd
126
127     ap = argparse.ArgumentParser(description="single-machine NumPy eALS")
128     ap.add_argument("--dataset", default="ml-100k")
129     ap.add_argument("--k", type=int, default=32)
130     ap.add_argument("--iters", type=int, default=20)
131     ap.add_argument("--alpha", type=float, default=0.4)
132     ap.add_argument("--c0", type=float, default=512.0)
133     ap.add_argument("--lam", type=float, default=0.01)
134     a = ap.parse_args()
135
136     proc = os.environ.get("EALS_PROC", os.path.expanduser("~/eals-spark/data/processed"))
137     df = pd.concat(pd.read_parquet(f) for f in glob.glob(f"{proc}/{a.dataset}/train/*.parquet"))
138     u = df.u.to_numpy(np.int64)
139     i = df.i.to_numpy(np.int64)
140     tr = []
141     t0 = time.perf_counter()
142     fit(u, i, int(u.max()) + 1, int(i.max()) + 1, K=a.k, lam=a.lam, c0=a.c0,
143         alpha=a.alpha, iters=a.iters, trace=tr)
144     print(f"{a.dataset}: |R|={len(u)} K={a.k} "
145         f" {(time.perf_counter()-t0)/a.iters:.3f} s/iteration")
146     for it, j in enumerate(tr, 1):
147         print(f" iter {it:3d} J = {j:.4f}")

```

A.4 src/eals_rdd.py - implémentation Spark RDD

```

1 """eALS on Spark RDDs.
2
3 Parallelisation is *exact*, not approximate. Section 4.2 of the paper: given Q
4 and  $S^q$ , the updates of the M user vectors touch disjoint parameters and read
5 only read-only state, so splitting users across workers reproduces the
6 sequential result up to the order of the sweep. Same for items given P,  $S^p$ .
7
8 One iteration:
9     1. broadcast Q and  $S^q$  (read-only for the whole user stage)
10    2. one task per user block -> new P rows + that block's partial  $P^T P$ 
11    3. reduce the partials into  $S^p$ , assemble P on the driver, broadcast it
12    4. one task per item block -> new Q rows + that block's partial sum  $c_i q_i q_i^T$ 
13    5. reduce into  $S^q$  for the next iteration
14
15 Steps 2/4 return the S partials for free, so the caches of Algorithm 1 lines 4
16 and 12 never need a separate pass over the factors.
17 """
18 import argparse
19 import json
20 import time
21
22 import numpy as np

```

```

23
24 from eals_local import item_confidence, update_items, update_users
25 from spark_utils import get_spark
26
27
28 def build_blocks(df, key, val, n_parts):
29     """One RDD element per partition holding the whole block as NumPy arrays:
30     (global ids of the rows, local row of each entry, column of each entry).
31
32     Built once and cached: rebuilding it every iteration would re-shuffle the
33     whole interaction matrix K times for nothing.
34     """
35     def to_arrays(it):
36         ks, vs = [], []
37         for k, v in it:
38             ks.append(k)
39             vs.append(v)
40         if not ks:
41             return iter(())
42         ks = np.asarray(ks, dtype=np.int64)
43         vs = np.asarray(vs, dtype=np.int64)
44         ids, rowid = np.unique(ks, return_inverse=True)
45         o = np.argsort(rowid, kind="stable")
46         return iter([(ids, rowid[o].astype(np.int64), vs[o])])
47
48     rdd = df.select(key, val).rdd.map(lambda r: (r[0], r[1]))
49     return rdd.partitionBy(n_parts).mapPartitions(to_arrays, preservesPartitioning=True)
50
51
52 def _user_task(blk, bcP, bcQ, bcC, lam, w, Sq):
53     ids, rowid, idx = blk
54     PT = np.ascontiguousarray(bcP.value[:, ids]) # this block's rows of P
55     update_users(PT, bcQ.value, idx, rowid, bcC.value, Sq, lam, w)
56     return ids, PT, PT @ PT.T # partial  $S^p = P_{blk}^T P_{blk}$ 
57
58
59 def _item_task(blk, bcP, bcQ, bcC, lam, w, Sp):
60     ids, rowid, idx = blk
61     QT = np.ascontiguousarray(bcQ.value[:, ids])
62     c_blk = bcC.value[ids]
63     update_items(QT, bcP.value, idx, rowid, c_blk, Sp, lam, w)
64     return ids, QT, (QT * c_blk) @ QT.T # partial  $S^q = \sum c_i q_i q_i^T$ 
65
66
67 def _objective_task(blk, bcP, bcQ, bcC, Sq, w):
68     """Eq. (7) through Eq. (14), split by user block."""
69     ids, rowid, idx = blk
70     PT = bcP.value[:, ids]
71     QT, c = bcQ.value, bcC.value
72     rhat = np.zeros(rowid.shape[0])
73     for f in range(PT.shape[0]):
74         rhat += PT[f][rowid] * QT[f][idx]
75     obs = float((w * (1.0 - rhat) ** 2).sum())
76     miss = float((PT * (Sq @ PT)).sum() - (c[idx] * rhat ** 2).sum())
77     return obs + miss
78
79
80 def fit(spark, train_df, M, N, K=32, lam=0.01, c0=512.0, alpha=0.4, iters=20,
81         partitions=8, seed=42, w=1.0, init_std=0.01, s_cache="fused",
82         eval_every=0, timing=None):
83     """Returns (P, Q, c, history). 'history' has one row per iteration with the
84     wall time of each phase, used by every scalability experiment."""
85     sc = spark.sparkContext
86     counts = np.zeros(N)
87     for i, n in train_df.groupby("i").count().collect():
88         counts[i] = n
89     cvec = item_confidence(counts, c0, alpha)
90     bcC = sc.broadcast(cvec)
91
92     ublocks = build_blocks(train_df, "u", "i", partitions).cache()
93     iblocks = build_blocks(train_df, "i", "u", partitions).cache()
94     ublocks.count(), iblocks.count() # force the shuffle once
95
96     rng = np.random.default_rng(seed)
97     PT = rng.normal(0, init_std, (K, M))

```

```

98 QT = rng.normal(0, init_std, (K, N))
99 Sq = (QT * cvec) @ QT.T
100 bcP = sc.broadcast(P)
101
102 hist = []
103 for it in range(iters):
104     t0 = time.perf_counter()
105     bcQ = sc.broadcast(Q)
106     Squ = Sq
107     t_bc1 = time.perf_counter()
108
109     res = ublocks.map(lambda b: _user_task(b, bcP, bcQ, bcC, lam, w, Squ)).collect()
110     t_u = time.perf_counter()
111     Sp = np.zeros((K, K))
112     for ids, blk, sp in res:
113         PT[:, ids] = blk
114         Sp += sp
115     if s_cache == "driver":
116         Sp = PT @ PT.T
117     bcP.unpersist()
118     bcP = sc.broadcast(P)
119     t_bc2 = time.perf_counter()
120
121     res = iblocks.map(lambda b: _item_task(b, bcP, bcQ, bcC, lam, w, Sp)).collect()
122     t_i = time.perf_counter()
123     Sq = np.zeros((K, K))
124     for ids, blk, sq in res:
125         QT[:, ids] = blk
126         Sq += sq
127     if s_cache == "driver":
128         Sq = (QT * cvec) @ QT.T
129     bcQ.unpersist()
130     t_end = time.perf_counter()
131
132     row = dict(iter=it, t_total=t_end - t0, t_user=t_u - t_bc1, t_item=t_i - t_bc2,
133               t_bcast=(t_bc1 - t0) + (t_bc2 - t_u), t_driver=(t_end - t_i))
134     if eval_every and (it + 1) % eval_every == 0:
135         bcQ2 = sc.broadcast(Q)
136         Sq2 = Sq
137         j = ublocks.map(lambda b: _objective_task(b, bcP, bcQ2, bcC, Sq2, w)).sum()
138         row["obj"] = j + lam * (float((PT ** 2).sum()) + float((QT ** 2).sum()))
139         bcQ2.unpersist()
140     hist.append(row)
141     if timing:
142         print(json.dumps(row))
143
144     ublocks.unpersist()
145     iblocks.unpersist()
146     bcP.unpersist()
147     bcC.unpersist()
148     return PT.T.copy(), QT.T.copy(), cvec, hist
149
150
151 if __name__ == "__main__":
152     from data import load_split
153
154     ap = argparse.ArgumentParser()
155     ap.add_argument("--dataset", default="ml-1m")
156     ap.add_argument("--k", type=int, default=32)
157     ap.add_argument("--cores", type=int, default=8)
158     ap.add_argument("--partitions", type=int, default=None)
159     ap.add_argument("--iters", type=int, default=10)
160     ap.add_argument("--alpha", type=float, default=0.4)
161     ap.add_argument("--c0", type=float, default=512.0)
162     ap.add_argument("--lam", type=float, default=0.01)
163     ap.add_argument("--eval-every", type=int, default=1)
164     a = ap.parse_args()
165
166     sp = get_spark(f"eals-{a.dataset}", cores=a.cores)
167     tr = load_split(sp, a.dataset, "train").cache()
168     M = tr.agg({"u": "max"}).collect()[0][0] + 1
169     N = tr.agg({"i": "max"}).collect()[0][0] + 1
170     P, Q, c, hist = fit(sp, tr, M, N, K=a.k, lam=a.lam, c0=a.c0, alpha=a.alpha,
171                       iters=a.iters, partitions=a.partitions or a.cores * 2,
172                       eval_every=a.eval_every, timing=True)

```

A.5 `src/evaluate.py` - $HR@k$, $NDCG@k$, classement sur catalogue entier

```

1  """Offline evaluation: leave-one-out ranking,  $HR@k$  and  $NDCG@k$ .
2
3  The held-out item is ranked against all items the user has not interacted
4  with in training -- the full protocol of the paper, not the cheap
5  "100 sampled negatives" variant, which is known to distort the ranking of
6  recommenders. Cost is  $O(M N K)$ , so it is done in Spark, block by block.
7  """
8  import numpy as np
9  from pyspark.sql import functions as F
10
11
12  def _block_eval(rows, bcP, bcQ, ks, block):
13      P, Q = bcP.value, bcQ.value          # (M, K) and (N, K)
14      N = Q.shape[0]
15      buf = []
16      out = []
17
18      def flush():
19          if not buf:
20              return
21          us = np.array([r[0] for r in buf], dtype=np.int64)
22          tis = np.array([r[2] for r in buf], dtype=np.int64)
23          S = P[us] @ Q.T                  # (b, N) BLAS matmul
24          for b, r in enumerate(buf):
25              tr = np.asarray(r[1], dtype=np.int64)
26              S[b, tr] = -np.inf           # never re-recommend
27          st = S[np.arange(len(buf)), tis].copy()
28          rank = (S > st[:, None]).sum(1) + 1    # 1-based rank of the GT item
29          for rk in rank:
30              out.append(tuple(
31                  [1.0 if rk <= k else 0.0 for k in ks]
32                  + [1.0 / np.log2(rk + 1) if rk <= k else 0.0 for k in ks]))
33          buf.clear()
34
35      for r in rows:
36          buf.append((r[0], r[1], r[2]))
37          if len(buf) >= block:
38              flush()
39              for o in out:
40                  yield o
41              out = []
42      flush()
43      for o in out:
44          yield o
45
46
47  def rank_metrics(spark, P, Q, train_df, test_df, ks=(10, 100), block=256):
48      """-> {'HR@10':..., 'NDCG@10':..., ...} averaged over test users."""
49      sc = spark.sparkContext
50      bcP, bcQ = sc.broadcast(np.ascontiguousarray(P)), sc.broadcast(np.ascontiguousarray(Q))
51
52      seen = train_df.groupBy("u").agg(F.collect_list("i").alias("seen"))
53      joined = test_df.select("u", F.col("i").alias("gt")).join(seen, "u", "left")
54      rdd = joined.rdd.map(lambda r: (r["u"], r["seen"] or [], r["gt"]))
55      n = rdd.count()
56      agg = (rdd.mapPartitions(lambda it: _block_eval(it, bcP, bcQ, ks, block))
57             .treeAggregate(np.zeros(2 * len(ks)),
58                           lambda a, b: a + np.asarray(b), lambda a, b: a + b))
59      bcP.unpersist(), bcQ.unpersist()
60      names = [f"HR@{k}" for k in ks] + [f"NDCG@{k}" for k in ks]
61      return dict(zip(names, (agg / n).tolist()))
62
63
64  def most_popular_factors(train_df, M, N):
65      """MostPopular expressed as a rank-1 MF model, so the trivial baseline goes
66      through exactly the same ranking code as eALS."""
67      cnt = np.zeros((N, 1))
68      for i, c in train_df.groupBy("i").count().collect():
69          cnt[i, 0] = c

```

```
69 return np.ones((M, 1)), cnt
```

A.6 src/baselines.py - ALS de MLlib et MostPopular

```
1 """Baselines: MLlib's implicit ALS (the Hu et al. 2008 algorithm eALS is
2 compared against in the paper) and MostPopular.
3
4 MLlib returns its factors as DataFrames; they are pulled back into dense NumPy
5 matrices so that eALS and ALS are scored by the very same ranking code.
6 """
7 import time
8
9 import numpy as np
10 from pyspark.ml.recommendation import ALS
11 from pyspark.sql import functions as F
12
13
14 def fit_mllib_als(train_df, M, N, K=32, lam=0.01, alpha=1.0, iters=10,
15                  seed=42, blocks=8):
16     t0 = time.perf_counter()
17     als = ALS(userCol="u", itemCol="i", ratingCol="r", implicitPrefs=True,
18              rank=K, regParam=lam, alpha=alpha, maxIter=iters, seed=seed,
19              numUserBlocks=blocks, numItemBlocks=blocks,
20              coldStartStrategy="drop", intermediateStorageLevel="MEMORY_AND_DISK")
21     m = als.fit(train_df.withColumn("r", F.lit(1.0)))
22     fit_s = time.perf_counter() - t0
23     P, Q = np.zeros((M, K)), np.zeros((N, K))
24     for r in m.userFactors.collect():
25         P[r["id"]] = r["features"]
26     for r in m.itemFactors.collect():
27         Q[r["id"]] = r["features"]
28     return P, Q, fit_s
29
30
31 def als_per_iter_seconds(train_df, M, N, K, lam, iters=11, blocks=8, seed=42):
32     """Marginal cost of one ALS iteration: fitting with 1 and with 'iters'
33     iterations and taking the slope cancels the fixed block-setup cost, which
34     would otherwise dominate on small data."""
35     _, _, t1 = fit_mllib_als(train_df, M, N, K, lam, iters=1, blocks=blocks, seed=seed)
36     _, _, tn = fit_mllib_als(train_df, M, N, K, lam, iters=iters, blocks=blocks, seed=
37                             seed)
38     return (tn - t1) / (iters - 1), t1, tn
```

A.7 src/check_correctness.py - le contrôle d'exactitude

```
1 """Correctness gate: the Spark implementation must produce the same model as the
2 sequential NumPy reference, whatever the number of blocks.
3
4 Users are independent given (Q, S^q) and items given (P, S^p), so blocking the
5 data across workers cannot change the result -- only the order in which the
6 floating-point sums are accumulated. Any deviation beyond ~1e-12 means a real
7 bug, not numerical noise.
8 """
9 import argparse
10
11 import numpy as np
12
13 import eals_local as L
14 import eals_rdd as R
15 from data import load_split
16 from spark_utils import get_spark
17
18
19 def brute_force_check(M=20, N=15, K=4, seed=7, lam=0.01, c0=64.0, alpha=0.4):
20     """The update rules (12)/(13) are supposed to be the exact coordinate
21     minimisers of the objective. Check that against Eq. (5)/(6) evaluated
22     literally over the *whole* M x N matrix -- O(MNK), only usable on a toy."""
23     rng = np.random.default_rng(seed)
24     R = (rng.random((M, N)) < 0.3).astype(float)
25     u_idx, i_idx = np.nonzero(R)
```

```

26 cvec = L.item_confidence(R.sum(0), c0, alpha)
27 PT = rng.normal(0, 0.3, (K, M))
28 QT = rng.normal(0, 0.3, (K, N))
29 W = np.where(R > 0, 1.0, cvec[None, :]) # wui
30
31 # Eq. (5) applied literally to every (u, f), no caching, no sparsity trick.
32 ref = PT.copy()
33 for uu in range(M):
34     for f in range(K):
35         rhat = QT.T @ ref[:, uu]
36         rf = rhat - ref[f, uu] * QT[f]
37         num = ((R[uu] - rf) * W[uu] * QT[f]).sum()
38         den = (W[uu] * QT[f] ** 2).sum() + lam
39         ref[f, uu] = num / den
40
41 fast = PT.copy()
42 urow, uidx = L.sort_by_row(u_idx, i_idx)
43 Sq = (QT * cvec) @ QT.T
44 L.update_users(fast, QT, uidx, urow, cvec, Sq, lam)
45 d_u = float(np.abs(fast - ref).max())
46
47 ref = QT.copy()
48 for ii in range(N):
49     for f in range(K):
50         rhat = PT.T @ ref[:, ii]
51         rf = rhat - ref[f, ii] * PT[f]
52         num = ((R[:, ii] - rf) * W[:, ii] * PT[f]).sum()
53         den = (W[:, ii] * PT[f] ** 2).sum() + lam
54         ref[f, ii] = num / den
55 fast = QT.copy()
56 irow, iidx = L.sort_by_row(i_idx, u_idx)
57 Sp = PT @ PT.T
58 L.update_items(fast, PT, iidx, irow, cvec, Sp, lam)
59 d_i = float(np.abs(fast - ref).max())
60 print(f"brute force vs Eq.(12): max|dP| = {d_u:.2e}")
61 print(f"brute force vs Eq.(13): max|dQ| = {d_i:.2e}")
62 return d_u < 1e-10 and d_i < 1e-10
63
64
65 def main(dataset="ml-100k", K=8, iters=5, cores=4):
66     ok_bf = brute_force_check()
67     print()
68     sp = get_spark("check", cores=cores)
69     tr = load_split(sp, dataset, "train").cache()
70     pdf = tr.toPandas()
71     u, i = pdf.u.to_numpy(np.int64), pdf.i.to_numpy(np.int64)
72     M, N = int(u.max()) + 1, int(i.max()) + 1
73     kw = dict(K=K, iters=iters, c0=512.0, alpha=0.4, lam=0.01)
74
75     Pl, Ql, cl = L.fit(u, i, M, N, **kw)
76     Jl = L.objective(np.ascontiguousarray(Pl.T), np.ascontiguousarray(Ql.T), u, i, cl,
77                     0.01)
78     rows = [("numpy-local", 1, 0.0, 0.0, Jl)]
79     for p in (1, 2, 4, 8):
80         P, Q, c, _ = R.fit(sp, tr, M, N, partitions=p, **kw)
81         J = L.objective(np.ascontiguousarray(P.T), np.ascontiguousarray(Q.T), u, i, c,
82                         0.01)
83         rows.append((f"spark-rdd", p, np.abs(P - Pl).max(), np.abs(Q - Ql).max(), J))
84
85     print(f"{'impl':<14}{'blocks':>7}{'max|dP|':>12}{'max|dQ|':>12}{'objective':>16}{'rel'
86           '.err':>12}")
87     for name, p, dp, dq, J in rows:
88         print(f"{'name':<14}{'p':>7}{'dp':>12.2e}{'dq':>12.2e}{'J':>16.6f}{'abs(J - Jl) / Jl':>12.2e}
89               ")
90     ok = ok_bf and all(r[2] < 1e-10 and r[3] < 1e-10 for r in rows)
91     print("\nRESULT:", "PASS" if ok else "FAIL")
92     sp.stop()
93     return ok
94
95 if __name__ == "__main__":
96     ap = argparse.ArgumentParser()
97     ap.add_argument("--dataset", default="ml-100k")
98     ap.add_argument("--k", type=int, default=8)
99     ap.add_argument("--iters", type=int, default=5)

```



```

97     ap.add_argument("--cores", type=int, default=4)
98     a = ap.parse_args()
99     raise SystemExit(0 if main(a.dataset, a.k, a.iters, a.cores) else 1)

```

A.8 src/experiments.py - la campagne de mesures

```

1  """All the measurements of the report. One sub-command per experiment, each
2  writing a raw CSV in results/ ; figures are produced separately by plots.py so
3  that a re-plot never needs a re-run.
4
5      python experiments.py strong --dataset ml-10m --k 32
6  """
7  import argparse
8  import csv
9  import json
10 import os
11 import time
12
13 import numpy as np
14
15 import baselines as B
16 import eals_rdd as R
17 import evaluate as EV
18 from data import load_split
19 from spark_utils import get_spark
20
21 RES = os.environ.get("EALS_RESULTS", os.path.expanduser("~/eals-spark/results"))
22 DEF = dict(K=32, lam=0.01, c0=512.0, alpha=0.4)
23
24
25 def write(name, rows):
26     os.makedirs(RES, exist_ok=True)
27     p = f"{RES}/{name}.csv"
28     with open(p, "w", newline="") as f:
29         w = csv.DictWriter(f, fieldnames=list(rows[0].keys()))
30         w.writeheader()
31         w.writerows(rows)
32     print(f"-> {p} ({len(rows)} rows)")
33     return p
34
35
36 def dims(df):
37     return (df.agg({"u": "max"}).collect()[0][0] + 1,
38             df.agg({"i": "max"}).collect()[0][0] + 1)
39
40
41 def per_iter(hist, warmup=1):
42     t = np.array([h["t_total"] for h in hist[warmup:]])
43     return dict(t_med=float(np.median(t)), t_min=float(t.min()), t_max=float(t.max()),
44                 t_user=float(np.median([h["t_user"] for h in hist[warmup:]])),
45                 t_item=float(np.median([h["t_item"] for h in hist[warmup:]])),
46                 t_bcast=float(np.median([h["t_bcast"] for h in hist[warmup:]])),
47                 t_driver=float(np.median([h["t_driver"] for h in hist[warmup:]])))
48
49
50 # ----- datasets
51 def exp_stats(a):
52     rows = []
53     for d in ("ml-100k", "ml-1m", "ml-10m", "amazon-movies"):
54         p = os.path.expanduser(f"~/eals-spark/data/processed/{d}/stats.json")
55         if os.path.exists(p):
56             rows.append(json.load(open(p)))
57     write("datasets", rows)
58
59
60 # ----- scalability
61 def exp_strong(a):
62     """(a) Strong scaling: same data, 1/2/4/8 cores.
63
64     With --interleave the repetitions are the *outer* loop, so a slow drift of
65     the machine (thermal throttling over a long campaign) hits every core count
66     equally instead of penalising whichever one runs last.
67     """

```

```

68 rows = []
69 cfigs = [(c, r) for r in range(a.reps) for c in (1, 2, 4, 8)] if a.interleave \
70 else [(c, r) for c in (1, 2, 4, 8) for r in range(a.reps)]
71 for cores, rep in cfigs:
72     if a.cooldown:
73         time.sleep(a.cooldown)
74     sp = get_spark(f"strong-{cores}", cores=cores)
75     tr = load_split(sp, a.dataset, "train").cache()
76     M, N = dims(tr)
77     # partitions proportional to cores: constant work per task.
78     nb = cores * a.pf
79     _, _, _, h = R.fit(sp, tr, M, N, iters=a.iters, partitions=nb, **DEF)
80     rows.append(dict(dataset=a.dataset, cores=cores, rep=rep,
81                     partitions=nb, **per_iter(h)))
82     sp.stop()
83     print(rows[-1])
84 write(f"strong_scaling{'_il' if a.interleave else ''}_{a.dataset}", rows)
85
86
87 def exp_factors(a):
88     """(b) The decisive experiment: how the cost of one iteration grows with K.
89
90     Two methodological details matter here, and we learned both the hard way.
91     Repeating a heavy configuration three times in a row heats the machine, so the
92     configurations that run last are penalised: --interleave makes the repetition
93     the outer loop, and --cooldown inserts an idle pause between configurations.
94     Without them two identical sweeps an hour apart differed by up to a factor two.
95     """
96     rows = []
97     sp = get_spark("factors", cores=a.cores)
98     tr = load_split(sp, a.dataset, "train").cache()
99     M, N = dims(tr)
100     ks = [int(k) for k in a.ks.split(",")]
101     jobs = []
102     for K in ks:
103         reps = a.reps if K <= a.reps_max_k else 1
104         jobs += [(K, r) for r in range(reps)]
105     if a.interleave:
106         jobs.sort(key=lambda x: (x[1], x[0]))
107     for K, rep in jobs:
108         if a.cooldown:
109             time.sleep(a.cooldown)
110         _, _, _, h = R.fit(sp, tr, M, N, iters=a.iters,
111                             partitions=a.cores * a.pf, **dict(DEF, K=K))
112         rows.append(dict(method="eALS-rdd", dataset=a.dataset, K=K, cores=a.cores,
113                         rep=rep, **per_iter(h)))
114     print(rows[-1])
115     if a.with_als:
116         for K in ks:
117             if K > a.als_max_k:
118                 continue
119             if a.cooldown:
120                 time.sleep(a.cooldown)
121             # The marginal cost of one ALS iteration is (T_n - T_1)/(n-1). With a
122             # small n the fixed block-setup cost dominates both terms and the
123             # estimate is pure noise (it can even come out negative), so n must be
124             # large enough for n-1 iterations to outweigh the setup.
125             t_it, t1, tn = B.als_per_iter_seconds(tr, M, N, K, DEF["lam"],
126                                                    iters=a.als_slope_iters,
127                                                    blocks=a.cores * a.pf)
128             rows.append(dict(method="MLlib-ALS", dataset=a.dataset, K=K, cores=a.cores,
129                             rep=0, t_med=t_it, t_min=t_it, t_max=t_it,
130                             t_user=0, t_item=0, t_bcast=0, t_driver=0))
131     print(rows[-1])
132     sp.stop()
133     write(f"factor_scaling_{a.dataset}", rows)
134
135
136 # ----- quality
137 def exp_quality(a):
138     """eALS vs MLlib ALS vs eALS(alpha=0) vs MostPopular, and accuracy as a
139     function of *wall-clock* time, not of iteration number."""
140     rows, curves = [], []
141     sp = get_spark("quality", cores=a.cores)
142     tr = load_split(sp, a.dataset, "train").cache()

```

```

143 te = load_split(sp, a.dataset, "test").cache()
144 M, N = dims(tr)
145 Pp, Qp = EV.most_popular_factors(tr, M, N)
146 rows.append(dict(method="MostPopular", dataset=a.dataset, K=0, fit_s=0.0,
147                 **EV.rank_metrics(sp, Pp, Qp, tr, te)))
148 print(rows[-1])
149
150 for name, alpha in (("eALS", DEF["alpha"]), ("eALS-uniform(alpha=0)", 0.0)):
151     d = dict(DEF, alpha=alpha, K=a.k)
152     P = Q = None
153     t_acc = 0.0
154     for step in a.als_iters:
155         P, Q, c, h = R.fit(sp, tr, M, N, iters=step,
156                             partitions=a.cores * a.pf, **d)
157         t_acc = sum(x["t_total"] for x in h)
158         m = EV.rank_metrics(sp, P, Q, tr, te)
159         curves.append(dict(method=name, dataset=a.dataset, iter=step,
160                             wall_s=t_acc, **m))
161     print(curves[-1])
162     rows.append(dict(method=name, dataset=a.dataset, K=a.k, fit_s=t_acc,
163                     **EV.rank_metrics(sp, P, Q, tr, te)))
164
165 for step in a.als_iters:
166     Pa, Qa, ts = B.fit_mllib_als(tr, M, N, K=a.k, lam=DEF["lam"], iters=step,
167                                   blocks=a.cores * a.pf)
168     m = EV.rank_metrics(sp, Pa, Qa, tr, te)
169     curves.append(dict(method="Mllib-ALS", dataset=a.dataset, iter=step,
170                         wall_s=ts, **m))
171     print(curves[-1])
172     rows.append(dict(method="Mllib-ALS", dataset=a.dataset, K=a.k, fit_s=ts, **m))
173 sp.stop()
174 write(f"quality_{a.dataset}", rows)
175 write(f"quality_curve_{a.dataset}", curves)
176
177
178 def exp_convergence(a):
179     rows = []
180     sp = get_spark("conv", cores=a.cores)
181     tr = load_split(sp, a.dataset, "train").cache()
182     te = load_split(sp, a.dataset, "test").cache()
183     M, N = dims(tr)
184     P, Q, c, h = R.fit(sp, tr, M, N, iters=a.iters, partitions=a.cores * a.pf,
185                         eval_every=1, **dict(DEF, K=a.k))
186     wall = 0.0
187     for it, x in enumerate(h):
188         wall += x["t_total"]
189         rows.append(dict(dataset=a.dataset, iter=it + 1, wall_s=wall, obj=x["obj"]))
190     write(f"convergence_{a.dataset}", rows)
191     sp.stop()
192
193
194 EXPS = dict(stats=exp_stats, strong=exp_strong, factors=exp_factors,
195             quality=exp_quality, convergence=exp_convergence)
196
197 if __name__ == "__main__":
198     ap = argparse.ArgumentParser()
199     ap.add_argument("exp", choices=sorted(EXPS))
200     ap.add_argument("--dataset", default="ml-10m")
201     ap.add_argument("--cores", type=int, default=8)
202     ap.add_argument("--pf", type=int, default=2, help="partitions per core")
203     ap.add_argument("--interleave", action="store_true",
204                     help="repetitions as the outer loop, to spread machine drift")
205     ap.add_argument("--iters", type=int, default=6)
206     ap.add_argument("--reps", type=int, default=3)
207     ap.add_argument("--k", type=int, default=32)
208     ap.add_argument("--ks", default="8,16,32,64,128,256")
209     ap.add_argument("--with-als", action="store_true")
210     ap.add_argument("--als-max-k", type=int, default=128)
211     ap.add_argument("--als-slope-iters", type=int, default=11)
212     ap.add_argument("--reps-max-k", type=int, default=64)
213     ap.add_argument("--cooldown", type=int, default=0,
214                     help="idle seconds between configurations, to limit thermal drift")
215     ap.add_argument("--als-iters", type=int, nargs="*", default=[1, 3, 5, 10, 20])
216     a = ap.parse_args()
217     EXPS[a.exp](a)

```

A.9 src/plots.py - figures

```

1  """Every figure of the report, regenerated from results/*.csv only.
2  Re-plotting never re-runs an experiment.
3
4      python plots.py          # all figures that have data
5  """
6  import os
7
8  import matplotlib
9  matplotlib.use("Agg")
10 import matplotlib.pyplot as plt
11 import numpy as np
12 import pandas as pd
13
14 RES = os.path.expanduser("~/eals-spark/results")
15 FIG = f"{RES}/figures"
16 plt.rcParams.update({"figure.dpi": 130, "font.size": 9, "axes.grid": True,
17                     "grid.alpha": .3, "legend.frameon": False,
18                     "savefig.bbox": "tight"})
19 C = {"eALS": "#1f77b4", "ALS": "#d62728"}
20
21
22 def load(name):
23     p = f"{RES}/{name}.csv"
24     return pd.read_csv(p) if os.path.exists(p) else None
25
26
27 def save(fig, name):
28     os.makedirs(FIG, exist_ok=True)
29     fig.savefig(f"{FIG}/{name}.pdf")
30     fig.savefig(f"{FIG}/{name}.png")
31     plt.close(fig)
32     print("->", name)
33
34
35 def slope(x, y):
36     """Exponent of the power law  $y = a x^b$ , fitted in log-log."""
37     return float(np.polyfit(np.log(x), np.log(y), 1)[0])
38
39
40 def amdahl_fit(p, t):
41     """ $T(p) = a + b/p \rightarrow$  serial fraction  $s = a/(a+b)$  with  $T(1) = a+b$ ."""
42     A = np.column_stack([np.ones_like(p, dtype=float), 1.0 / p])
43     a, b = np.linalg.lstsq(A, t, rcond=None)[0]
44     return max(0.0, min(1.0, a / (a + b))), a, b
45
46
47 def fig_strong():
48     runs = []
49     for ds in ("ml-1m", "ml-10m"):
50         for tag in ("_il", ""):
51             d = load(f"strong_scaling{tag}_{ds}")
52             # the interleaved run supersedes the plain one when both exist
53             if d is None or (tag == "" and load(f"strong_scaling_il_{ds}") is not None):
54                 continue
55             runs.append((f"{ds}, $B=2p$", d))
56     if not runs:
57         return
58     fig, ax = plt.subplots(1, 3, figsize=(11.5, 3.3))
59     for name, d in runs:
60         g = d.groupby("cores").t_med
61         med, lo, hi = g.median(), g.min(), g.max()
62         p = med.index.to_numpy(float)
63         ax[0].errorbar(p, med, yerr=[med - lo, hi - med], marker="o",
64                        capsize=3, label=name)
65         sp = med.iloc[0] / med
66         s, _, _ = amdahl_fit(p, med.to_numpy())
67         ax[1].plot(p, sp, marker="o", label=f"{name} ($s={s:.3f}$)")
68         ax[2].plot(p, sp / p, marker="o", label=name)
69     pp = np.array([1, 2, 4, 8])
70     ax[1].plot(pp, pp, ":", c="k", lw=1.2, label="ideal")
71     ax[2].axhline(1, ls=":", c="k", lw=1.2)
72     for a, t, yl in zip(ax, ("time per iteration", "speedup  $S(p)=T_1/T_p$ ",
73                             "efficiency  $E(p)=S(p)/p$ "), ("seconds", r"$\times$", "")):
74

```

```

74     a.set_xlabel("Spark cores $p$ (local[p])")
75     a.set_title(t)
76     a.set_ylabel(yl)
77     a.set_xscale("log", base=2)
78     a.set_xticks(pp), a.set_xticklabels(pp)
79     a.legend(fontsize=6.5)
80 ax[0].set_yscale("log")
81 save(fig, "strong_scaling")
82
83
84 def fig_factors():
85     dss = [d for d in ("amazon-movies", "ml-1m") if load(f"factor_scaling_{d}") is not
86             None]
87     if not dss:
88         return
89     fig, ax = plt.subplots(1, len(dss), figsize=(4.9 * len(dss), 3.5), squeeze=False)
90     for a, ds in zip(ax[0], dss):
91         d = load(f"factor_scaling_{ds}")
92         ref = None
93         for m, sub in d.groupby("method"):
94             g = sub[sub.t_med > 0].groupby("K").t_med.median()
95             if len(g) < 2:
96                 continue
97             b = slope(g.index.to_numpy(), g.to_numpy())
98             hi = g[g.index >= g.index.max() / 4] # top two octaves
99             bh = slope(hi.index.to_numpy(), hi.to_numpy()) if len(hi) > 1 else b
100            a.plot(g.index, g.values, marker="o",
101                  c=C["eALS"] if "eALS" in m else C["ALS"],
102                  label=f"{m}: slope {b:.2f} (top range {bh:.2f})")
103            if "eALS" in m:
104                ref = g
105            lo, hi_ = d[d.t_med > 0].t_med.min(), d.t_med.max()
106            if ref is not None:
107                k = np.array([ref.index.min(), ref.index.max()], dtype=float)
108                for e, st in ((1, "-."), (2, "--"), (3, ":")):
109                    a.plot(k, ref.iloc[0] * (k / k[0]) ** e, st, c="grey", lw=1,
110                          label=f"$K^{e}$")
111            a.set_ylim(lo / 2, hi_ * 3) # keep the guides from rescaling the plot
112            a.set_xscale("log", base=2), a.set_yscale("log")
113            a.set_xlabel("number of latent factors K")
114            a.set_ylabel("seconds / iteration")
115            a.set_title(f"{ds}, 8 cores")
116            a.legend(fontsize=6.5)
117        save(fig, "factor_scaling")
118
119 def fig_convergence():
120     if all(load(f"convergence_{d}") is None for d in ("ml-1m", "amazon-movies")):
121         return
122     fig, ax = plt.subplots(1, 2, figsize=(8.4, 3.2))
123     for j, ds in enumerate(("ml-1m", "amazon-movies")):
124         d = load(f"convergence_{ds}")
125         if d is None:
126             continue
127         ax[j].plot(d.iter, d.obj, marker=".", c=C["eALS"])
128         ax[j].set_xlabel("iteration"), ax[j].set_ylabel("objective $J$")
129         ax[j].set_title(f"{ds} (K=32, $c_0$=512, $\alpha$=0.4)")
130         ax[j].set_yscale("log")
131     save(fig, "convergence")
132
133
134 def fig_quality_time():
135     if all(load(f"quality_curve_{d}") is None for d in ("ml-1m", "amazon-movies")):
136         return
137     fig, ax = plt.subplots(1, 2, figsize=(8.6, 3.3))
138     for j, ds in enumerate(("ml-1m", "amazon-movies")):
139         d = load(f"quality_curve_{ds}")
140         if d is None:
141             continue
142         for m, sub in d.groupby("method"):
143             sub = sub.sort_values("wall_s")
144             ax[j].plot(sub.wall_s, sub["HR@100"], marker="o", ms=3, label=m)
145         ax[j].set_xlabel("wall-clock training time (s)")
146         ax[j].set_ylabel("HR@100")
147         ax[j].set_title(ds)

```

```

148         ax[j].set_xscale("log")
149         ax[j].legend(fontsize=7)
150     save(fig, "quality_vs_time")
151
152
153 if __name__ == "__main__":
154     for f in (fig_strong, fig_factors, fig_convergence, fig_quality_time):
155         try:
156             f()
157         except Exception as e:           # a missing experiment must not stop the rest
158             print(f"skip {f.__name__}: {type(e).__name__}: {e}")

```

A.10 report/tables.py - chaque chiffre de ce rapport

```

1  """Generates report/tables/*.tex and report/tables/macros.tex from results/*.csv.
2
3  Every number quoted in the report comes from here, so the text can never drift
4  away from the measurements.
5  """
6  import os
7
8  import numpy as np
9  import pandas as pd
10
11 RES = os.path.expanduser("~/eals-spark/results")
12 OUT = os.path.dirname(os.path.abspath(__file__)) + "/tables"
13 os.makedirs(OUT, exist_ok=True)
14 # Every macro the report may reference. tables.py overwrites the ones whose
15 # experiment actually ran; the rest keep a visible placeholder so that a partial
16 # campaign still compiles instead of failing on an undefined control sequence.
17 MAC = {k: "\\textbf{[n/a]}" for k in (
18     "CorrWorst "
19     "SpeedupBig SerialBig ToneBig TeightBig SpeedupSmall SerialSmall ToneSmall
20     TeightSmall "
21     "SlopeEals SlopeAls "
22     "SlopeEalsAmz SlopeAlsAmz SlopeHiEalsAmz SlopeHiAlsAmz "
23     "SlopeEalsMl SlopeAlsMl SlopeHiEalsMl SlopeHiAlsMl CrossKAmz CrossKMl "
24     "ConvItersAmz ConvItersMl AlsBestHR AlsTimeToBest EalsTimeToAls EalsOverAls "
25     "EalsOverAlsBest PopGain PopFloor PopGainMl PopFloorMl EalsOverAlsMl "
26     "HRe HRa HRu HReMl HRaMl HRuMl "
27     "FitConstEalsAmz FitRtwoEalsAmz FitCrossEalsAmz FitConstAlsAmz FitRtwoAlsAmz "
28     "FitCrossAlsAmz FitConstEalsMl FitRtwoEalsMl FitCrossEalsMl FitConstAlsMl "
29     "FitRtwoAlsMl FitCrossAlsMl RatioAtMaxAmz KAtRatioAmz RatioAtMinAmz "
30     "KAtRatioMinAmz FlipKAmz GrowthEalsAmz GrowthAlsAmz GrowthEalsMl GrowthAlsMl "
31     "RatioAtMaxMl KAtRatioMl RatioAtMinMl KAtRatioMinMl FlipKMl").split()}
32
33 def load(n):
34     p = f"{RES}/{n}.csv"
35     return pd.read_csv(p) if os.path.exists(p) else None
36
37
38 def put(name, body):
39     open(f"{OUT}/{name}.tex", "w").write(body.rstrip("\n") + "\n")
40     print("->", name)
41
42
43 def mac(k, v):
44     MAC[k] = v
45
46
47 def missing(name, what):
48     put(name, "\\multicolumn{1}{1}{\\emph{experiment \\texttt{%s} not run}} \\\\n"
49         % what.replace("_", "\\_"))
50
51
52 def t_datasets():
53     d = load("datasets")
54     if d is None:
55         return missing("tab_datasets", "stats")
56     rows = []
57     for _, r in d.iterrows():
58         rows.append(f"{r['dataset']} & {int(r['users']):,} & {int(r['items']):,} & "

```

```

59         f"{int(r['interactions']):,} & {100*r['sparsity']:.2f}\\% & "
60         f"{int(r['train']):,} & {int(r['test']):,} \\\\"")
61     put("tab_datasets", "\n".join(rows).replace(",", "\\,"))
62
63
64 def t_correctness():
65     p = f"{RES}/correctness.txt"
66     if not os.path.exists(p):
67         return missing("tab_correctness", "check_correctness")
68     rows, worst = [], 0.0
69     for ln in open(p).read().splitlines()[1:]:
70         f = ln.split()
71         if len(f) != 6 or not f[1].isdigit():
72             continue
73         rows.append(f"\\texttt{{{f[0]}}} & {f[1]} & {f[2]} & {f[3]} & {f[4]} & {f[5]} \\\\"")
74         worst = max(worst, float(f[2]), float(f[3]))
75     put("tab_correctness", "\n".join(rows))
76     mac("CorrWorst", f"{worst:.0e}".replace("e-", "\\cdot 10^{-"} + ")")
77
78
79 def _amdahl(p, t):
80     A = np.column_stack([np.ones_like(p, dtype=float), 1.0 / p])
81     a, b = np.linalg.lstsq(A, t, rcond=None)[0]
82     return max(0.0, min(1.0, a / (a + b)))
83
84
85 def t_strong():
86     body = []
87     for ds in ("ml-1m", "ml-10m"):
88         for tag in ("_il", ""):
89             d = load(f"strong_scaling_{tag}_{ds}")
90             if d is None or (tag == "" and load(f"strong_scaling_il_{ds}") is not None):
91                 continue
92             g = d.groupby("cores").t_med
93             med, lo, hi = g.median(), g.min(), g.max()
94             p = med.index.to_numpy(float)
95             sp = med.iloc[0] / med
96             s = _amdahl(p, med.to_numpy())
97             body.append(f"\\multicolumn{5}{{1}}{\\textit{{%s, B = 2p}} - fitted Amdahl "
98                 "serial fraction $s=\\%.3f$}\\\\" % (ds, s))
99             for c in med.index:
100                 body.append(f"\\quad {{c}} & {{med[c]:.3f}} & {{lo[c]:.3f}}--{{hi[c]:.3f}} & "
101                     f"{{sp[c]:.2f}} & {{100*sp[c]/c:.0f}}\\% \\\\"")
102             k = "Big" if ds == "ml-10m" else "Small"
103             mac("Speedup" + k, f"{{sp.iloc[-1]:.2f}}")
104             mac("Serial" + k, f"{{s:.3f}}")
105             mac("Tone" + k, f"{{med.iloc[0]:.2f}}")
106             mac("Teight" + k, f"{{med.iloc[-1]:.2f}}")
107     if not body:
108         return missing("tab_strong", "strong")
109     put("tab_strong", "\n".join(body))
110
111
112 def _poly_fit(K, t, powers):
113     """Least-squares fit of  $t = a + \sum_p b_p K^p$ , returning (a, {p: b_p}, R^2)."""
114     K = np.asarray(K, float)
115     A = np.column_stack([np.ones_like(K)] + [K ** p for p in powers])
116     c = np.linalg.lstsq(A, t, rcond=None)[0]
117     pred = A @ c
118     ss = 1 - ((t - pred) ** 2).sum() / ((t - t.mean()) ** 2).sum()
119     return c[0], dict(zip(powers, c[1:])), ss
120
121
122 def t_factors():
123     dss = [d for d in ("amazon-movies", "ml-1m") if load(f"factor_scaling_{d}") is not None]
124     if not dss:
125         return missing("tab_factors", "factors")
126     rows = []
127     for ds in dss:
128         d = load(f"factor_scaling_{ds}")
129         piv = d[d.t_med > 0].groupby(["K", "method"]).t_med.median().unstack()
130         rows.append(f"\\multicolumn{4}{{1}}{\\textit{{%s}}}\\\\" % ds)
131         for K, r in piv.iterrows():

```

```

132     e = r.get("eALS-rdd", float("nan"))
133     a = r.get("MLlib-ALS", float("nan"))
134     rows.append(f"\quad {K} & {e:.2f} & " +
135                (f"{a:.2f} & {a/e:.1f}$\\times$ \\\\" if a == a
136                 else "- & - \\\\"))
137     key = "Amz" if ds == "amazon-movies" else "M1"
138     # The theory says eALS costs a + b/R/K + c(M+N)K^2 and ALS a + bK^2 + cK^3.
139     # Fitting those forms is more informative than a single log-log slope, and it
140     # gives the K at which the higher-order term takes over.
141     for m, mk, pw in (("eALS-rdd", "Eals", (1, 2)), ("MLlib-ALS", "Als", (2, 3))):
142         if m not in piv:
143             continue
144         g = piv[m].dropna()
145         if len(g) < 4:
146             continue
147         a0, b, r2 = _poly_fit(g.index.to_numpy(), g.to_numpy(), pw)
148         lo, hi = pw
149         mac(f"FitConst{mk}{key}", f"{a0:.2f}")
150         mac(f"FitRtwo{mk}{key}", f"{r2:.4f}")
151         if b[hi] > 0 and b[lo] > 0:
152             mac(f"FitCross{mk}{key}", f"{b[lo]/b[hi]:.0f}")
153     for m, mk in (("eALS-rdd", "Eals"), ("MLlib-ALS", "Als")):
154         if m not in piv:
155             continue
156         g = piv[m].dropna()
157         mac(f"Slope{mk}{key}",
158            f"{np.polyfit(np.log(g.index), np.log(g.values), 1)[0]:.2f}")
159         hi = g[g.index >= g.index.max() / 4]
160         if len(hi) > 1:
161             mac(f"SlopeHi{mk}{key}",
162                f"{np.polyfit(np.log(hi.index), np.log(hi.values), 1)[0]:.2f}")
163     put("tab_factors", "\n".join(rows))
164     for ds, key in (("amazon-movies", "Amz"), ("ml-1m", "M1")):
165         d = load(f"factor_scaling_{ds}")
166         if d is None:
167             continue
168         piv = d[d.t_med > 0].groupby(["K", "method"]).t_med.median().unstack()
169         if "MLlib-ALS" in piv and "eALS-rdd" in piv:
170             r = (piv["MLlib-ALS"] / piv["eALS-rdd"]).dropna()
171             mac("RatioAtMax" + key, f"{r.iloc[-1]:.1f}")
172             mac("KAtRatio" + key, f"{r.index[-1]:g}")
173             mac("RatioAtMin" + key, f"{1/r.iloc[0]:.1f}")
174             mac("KAtRatioMin" + key, f"{r.index[0]:g}")
175             fl = r[r > 1]
176             if len(fl):
177                 mac("FlipK" + key, f"{fl.index[0]:g}")
178             # growth factor over a fixed K window: robust to the additive constant and
179             # to the implementation language, unlike a fitted exponent
180             for m, mk in (("eALS-rdd", "Eals"), ("MLlib-ALS", "Als")):
181                 if m in piv and 32 in piv.index and 128 in piv.index:
182                     g = piv[m]
183                     if g.get(32) == g.get(32) and g.get(128) == g.get(128):
184                         mac("Growth" + mk + key, f"{g[128]/g[32]:.1f}")
185             # the report quotes the Amazon numbers as the headline ones
186             for a, b in (("SlopeEals", "SlopeHiEalsAmz"), ("SlopeAls", "SlopeHiAlsAmz")):
187                 if b in MAC and "n/a" not in MAC[b]:
188                     mac(a, MAC[b])
189
190
191 def t_crossk():
192     ""K* = 2|R|/(M+N): above it the (M+N)K^2 term of eALS overtakes the
193     |R|K term, i.e. this is where the quadratic regime actually starts."""
194     d = load("datasets")
195     if d is None:
196         return
197     for ds, key in (("amazon-movies", "Amz"), ("ml-1m", "M1")):
198         r = d[d.dataset == ds]
199         if len(r):
200             r = r.iloc[0]
201             mac("CrossK" + key,
202                f"{2*r['train']/(r['users']+r['items']):.0f}")
203
204
205 def t_convergence():
206     ""How many iterations to get within 1% of the objective at 30 iterations."""

```



```

207 rows = []
208 for ds in ("ml-1m", "amazon-movies"):
209     d = load(f"convergence_{ds}")
210     if d is None:
211         continue
212     d = d.sort_values("iter")
213     jf = d.obj.iloc[-1]
214     k = int(d[d.obj <= 1.01 * jf].iter.min())
215     rows.append(f"{ds} & {d.obj.iloc[0]:.0f} & {jf:.0f} & "
216               f"{100*(1 - jf/d.obj.iloc[0]):.1f}% & {k} & "
217               f"{d[d.iter == k].wall_s.iloc[0]:.1f} \\\\")
218     mac("ConvIters" + ("Amz" if "amazon" in ds else "Ml"), str(k))
219 if not rows:
220     return missing("tab_convergence", "convergence")
221 put("tab_convergence", "\n".join(rows))
222
223
224 def t_qualitytime():
225     """How long eALS needs to reach the best accuracy MLlib ALS ever gets to."""
226     d = load("quality_curve_amazon-movies")
227     if d is None:
228         return
229     als = d[d.method == "MLlib-ALS"]
230     ea = d[d.method == "eALS"].sort_values("wall_s")
231     if not len(als) or not len(ea):
232         return
233     best = als["HR@100"].max()
234     mac("AlsBestHR", f"{best:.4f}")
235     mac("AlsTimeToBest", f"{als.loc[als['HR@100'].idxmax()].wall_s:.1f}")
236     hit = ea[ea["HR@100"] >= best]
237     if len(hit):
238         mac("EalsTimeToAls", f"{hit.wall_s.iloc[0]:.1f}")
239     mac("EalsOverAlsBest", f"{100*(ea['HR@100'].max()-best - 1):.0f}")
240
241
242 def t_quality():
243     rows = []
244     for ds in ("ml-1m", "amazon-movies"):
245         d = load(f"quality_{ds}")
246         if d is None:
247             continue
248         rows.append("\\multicolumn{6}{l}{\\textit{%s}}\\\\\\ % ds)
249         d = d.drop_duplicates("method", keep="last")
250         for _, r in d.iterrows():
251             rows.append(f"\\quad {r.method} & {r['HR@10']:.4f} & {r['HR@100']:.4f} & "
252                       f"{r['NDCG@10']:.4f} & {r['NDCG@100']:.4f} & {r['fit_s']:.1f} \\\\")
253         g = d.set_index("method")["HR@100"]
254         k = "" if ds == "amazon-movies" else "Ml"
255         if "eALS" in g and "eALS-uniform(alpha=0)" in g:
256             mac("PopGain" + k, f"{100*(g['eALS']/g['eALS-uniform(alpha=0)']-1):.1f}")
257         if "eALS" in g and "MostPopular" in g:
258             mac("PopFloor" + k, f"{g['eALS']/g['MostPopular']:.1f}")
259         if "eALS" in g and "MLlib-ALS" in g:
260             mac("EalsOverAls" + k, f"{100*(g['eALS']/g['MLlib-ALS']-1):.0f}")
261             mac("HRe" + k, f"{g['eALS']:.4f}")
262             mac("HRa" + k, f"{g['MLlib-ALS']:.4f}")
263             mac("HRu" + k, f"{g['eALS-uniform(alpha=0)']:.4f}")
264     if not rows:
265         return missing("tab_quality", "quality")
266     put("tab_quality", "\n".join(rows))
267
268
269 if __name__ == "__main__":
270     for f in (t_datasets, t_correctness, t_strong, t_factors, t_crossk,
271             t_convergence, t_qualitytime, t_quality):
272         try:
273             f()
274         except Exception as e:
275             print(f"skip {f.__name__}: {type(e).__name__}: {e}")
276     body = "\n".join(f"\\newcommand{{{k}}}{{{v}}}" for k, v in sorted(MAC.items()))
277     open(f"{OUT}/macros.tex", "w").write(body + "\n")
278     print(f"-> macros.tex ({len(MAC)} macros)")
279     for k, v in sorted(MAC.items()):
280         print(f"    \\{k} = {v}")

```

A.11 scripts/env.sh, run_all.sh, build_report.sh

```

1 # Spark needs a JDK it supports and macOS ships none: point JAVA_HOME at the
2 # Homebrew JDK before anything else. 'source scripts/env.sh' from the repo root.
3 export JAVA_HOME=${JAVA_HOME:-/opt/homebrew/opt/openjdk@17}
4 export PATH="$JAVA_HOME/bin:$PATH"
5 export PYSARK_PYTHON=${PYSARK_PYTHON:-$(command -v python3)}
6 export PYSARK_DRIVER_PYTHON=$PYSARK_PYTHON
7 export EALS_SPARK_TMP=${EALS_SPARK_TMP:-/tmp/eals-spark-tmp}
8 # NumPy is already the inner parallelism of one Spark task; letting BLAS spawn
9 # its own threads on top of local[k] oversubscribes the 8 cores and makes every
10 # scalability measurement meaningless.
11 export OMP_NUM_THREADS=1
12 export OPENBLAS_NUM_THREADS=1
13 export MKL_NUM_THREADS=1
14 export VECLIB_MAXIMUM_THREADS=1
15 export NUMEXPR_NUM_THREADS=1

```

```

1 #!/usr/bin/env bash
2 # Full experimental campaign, in order, one experiment at a time.
3 #
4 # Timing experiments must never overlap: they share the 8 cores of the machine, and a
5 # laptop under sustained load throttles. Two rules follow from that and are applied
6 # below:
7 #   --interleave puts the repetition in the outer loop, so a slow drift over the
8 #               campaign hits every configuration equally instead of penalising
9 #               whichever one runs last;
10 #   --cooldown N idles N seconds between configurations.
11 # Budget: about 1 h on an 8-core MacBook. Raw CSVs land in results/.
12 set -u
13 cd "$(dirname "$0")/.." || exit 1
14 source scripts/env.sh
15 cd src
16 LOG=./results/run_all.log
17 run(){ echo "==== $* @ $(date +%H:%M:%S) =====" | tee -a $LOG
18       python3 -u experiments.py "$@" >>$LOG 2>/dev/null || echo "FAILED: $*" | tee -a
19       $LOG; }
20 echo "campaign started $(date)" > $LOG
21
22 # --- correctness first: nothing else means anything if this fails -----
23 python3 check_correctness.py --dataset ml-100k --k 8 --iters 5 --cores 4 \
24     2>/dev/null | tee ../results/correctness.txt
25 run stats
26
27 # --- scalability -----
28 run strong --dataset ml-10m --iters 4 --reps 3 --interleave
29 run strong --dataset ml-1m --iters 4 --reps 3 --interleave --cooldown 15
30 run factors --dataset amazon-movies --cores 8 --iters 4 --reps 3 --reps-max-k 256 \
31     --ks 8,16,32,64,128,256 --with-als --als-max-k 128 --interleave --
32     cooldown 20
33 run factors --dataset ml-1m --cores 8 --iters 4 --reps 3 --reps-max-k 256 \
34     --ks 8,16,32,64,128,256 --with-als --als-max-k 128 --interleave --
35     cooldown 20
36
37 # --- accuracy -----
38 run convergence --dataset amazon-movies --cores 8 --iters 30 --k 32
39 run convergence --dataset ml-1m --cores 8 --iters 30 --k 32
40 run quality --dataset amazon-movies --cores 8 --k 32 --als-iters 1 2 3 5 8 12 20 30
41 run quality --dataset ml-1m --cores 8 --k 32 --als-iters 1 2 3 5 8 12 20 30
42
43 echo "campaign finished $(date)" | tee -a $LOG
44 echo "now run ./scripts/build_report.sh"

```

```

1 #!/usr/bin/env bash
2 # results/*.csv -> figures -> LaTeX tables and macros -> report.pdf
3 set -eu
4 cd "$(dirname "$0")/.."
5 source scripts/env.sh
6 python3 src/plots.py
7 python3 report/tables.py
8 cd report
9 pdflatex -interaction=nonstopmode report.tex >/dev/null
10 bibtex report >/dev/null || true

```

```

11 pdflatex -interaction=nonstopmode report.tex >/dev/null
12 pdflatex -interaction=nonstopmode report.tex >/dev/null
13 cp report.pdf ../Rapport_Sinapi_Ernadote_eALS_Spark.pdf
14 echo "Rapport_Sinapi_Ernadote_eALS_Spark.pdf: $(pdftinfo report.pdf | awk '/Pages/{print $2}') pages"
15 grep -c '\[n/a\]' tables/macros.tex | xargs -I{} echo "{} macro(s) still without data"

```

B Le notebook de démonstration

`notebooks/demo_small_data.ipynb` exécute tout le pipeline de bout en bout sur **ml-100k** (943 utilisateurs, 1 152 items, 97 953 interactions) en quelques minutes sur un portable. Son but est la vérification, pas la mesure : un lecteur devrait pouvoir exécuter « Run All » et voir, sans attendre, que chaque affirmation de ce rapport est étayée par du code qui s'exécute. Chaque section indique son entrée et sa sortie.

Table 9: Cellules du notebook, avec entrée et sortie.

§	Entrée	Sortie	Ce que cela démontre
1	le JDK installé	versions Java / Spark / NumPy, master Spark et parallélisme	l'environnement est bien celui décrit en Section 2.4 ; en particulier <code>JAVA_HOME</code> pointe vers un JDK 17.
2	<code>data/raw/ml-100k/u.data</code> (brut <code>user item ratings</code>)	le dictionnaire de statistiques et les premières lignes du découpage d'entraînement	le filtrage 10-core, la ré-indexation dense et le découpage leave-one-out reproduisent le Tableau 2, ligne ml-100k .
3	fréquences des items d'entraînement	min/médiane/max de c_i et $\sum_i c_i$	l'Éq. (3) est correctement implémentée : $\sum_i c_i = c_0$ pour tout α , et $\alpha = 0,4$ étale c_i sur un rapport 6:1 sur ml-100k alors que $\alpha = 0$ le réduit à c_0/N . Cela montre aussi que l'item le plus populaire atteint $c_i > 1$ - la réserve S2 de la Section 6.
4	les paires d'entraînement sous forme de deux tableaux d'entiers	un tableau de J par itération et sa décroissance	l'objectif (2) décroît de façon monotone : les règles de mise à jour (9)–(10) sont correctes.
5	le DataFrame d'entraînement, 8 partitions	temps réel par itération décomposé en étape utilisateur / étape item / broadcast / driver	l'anatomie d'une itération distribuée, et la part prise par la partie non parallèle.
6	les modèles des §4 et §5	$\max \Delta P $, $\max \Delta Q $ et une assertion (<code>assert</code>)	le contrôle d'exactitude de la Section 5.1, en miniature. La cellule <i>échoue bruyamment</i> si les implémentations divergent.
7	matrices de facteurs + l'ensemble de test leave-one-out	HR@10/@100 et NDCG@10/@100 pour MostPopular, eALS, eALS avec $\alpha = 0$ et MLlib ALS	eALS bat le plancher de popularité avec une large marge, et $\alpha > 0$ bat $\alpha = 0$.
8	les mêmes données réajustées avec 1, 2, 4 cœurs	secondes par itération, accélération, efficacité	une miniature de la Section 5.2.1, et une honnête : sur ml-100k l'accélération ressort <i>en dessous de un</i> - plus de cœurs rendent eALS plus lent, car les données sont bien trop petites pour Spark.

Temps d'exécution. Environ deux minutes de bout en bout sur la machine de la Section 2.4, dominées par les démarrages de session Spark (§8 redémarre la session une fois par nombre de cœurs,

ce qui est inévitable : `local[p]` est fixé à la création de la session).